

ML Defender (aRGus NDR): An Open-Source Embedded ML NIDS for Botnet and Anomalous Traffic Detection in Resource-Constrained Organizations

Alonso Isidoro Román
Independent Researcher, Extremadura, Spain
github.com/alonsoir/argus

Draft v19 — May 2026

Abstract

Ransomware and DDoS attacks disproportionately impact hospitals, schools, and small organizations that cannot afford enterprise security solutions. Existing open-source alternatives either rely on signature matching — missing novel variants — or require infrastructure beyond the reach of their target users. We present **ML Defender (aRGus NDR)**, an open-source network intrusion detection system built in C++20 that brings embedded machine learning inference to organizations with limited resources, deployable on commodity bare-metal hardware at approximately 150–200 USD fully dedicated.

ML Defender implements a six-component pipeline over eBPF/XDP packet capture, ZeroMQ transport, and Protocol Buffers serialization, terminating in a dual-score detection architecture combining a rule-based Fast Detector with an embedded Random Forest classifier. The *Maximum Threat Wins* policy selects the arithmetic maximum of both detector scores, using ML inference to suppress false positives generated by the heuristic layer.

We evaluate ML Defender against the CTU-13 Neris botnet dataset, achieving $F1 = 0.9985$, $Precision = 0.9969$, and $Recall = 1.0000$, with a false positive rate of 0.0002% (2 FP in 12,075 benign flows) — the two false positives identified as host-only adapter artifacts: an mDNS multicast frame (192.168.56.1 → 224.0.0.251) and a broadcast frame (192.168.56.1 → 192.168.56.255), both generated by the VirtualBox host-only NIC and absent from production network topologies. Whether these artifacts appear in bare-metal deployments remains unverified; bare-metal characterization is Future Work (§11). The Fast Detector alone produces a FPR of 6.61% on purely benign traffic (bigFlows); the ML Detector reduces real production blocks to zero on the same corpus, an approximately 500-fold reduction. Per-class inference latency ranges from 0.24 μ s (DDoS) to 1.06 μ s (Ransomware), enabling real-time detection on commodity hardware.

We further characterize the pipeline’s throughput ceiling under progressive load in a virtualized environment. Requesting up to 100 Mbps via `tcpreplay` against the VirtualBox NIC, the pipeline processes up to ~34–38 Mbps (the emulated NIC ceiling) with zero packet drops and zero errors across 2.37 million packets. At peak load, `ml-detector` consumes approximately 3.2 CPU cores; total pipeline RAM remains stable at ~1.28 GB with negligible drift (~18 MB over 8 minutes). The bottleneck is VirtualBox NIC emulation, not pipeline

logic. Post-replay drain behavior confirms that ZeroMQ queues absorb burst traffic correctly and drain asynchronously, validating the queue stability design. All throughput figures should be treated as conservative lower bounds; bare-metal characterization remains future work (§11).

These results demonstrate architectural feasibility under controlled replay conditions rather than universal detection capability. The evaluation covers a single botnet scenario from 2011 with synthetic-trained classifiers; generalizability to modern ransomware, DDoS variants, and encrypted C2 traffic has not been empirically established and is discussed in Section 10. Behavioral indicators associated with ransomware propagation and lateral-movement patterns are detected in the Neris scenario; direct evaluation against post-2020 ransomware families remains Future Work §11.

All experiments were conducted in a reproducible Vagrant/VirtualBox environment running on a consumer laptop, ensuring result accessibility and independent reproducibility.

This work was developed through a structured multi-model AI collaboration methodology — the *Consejo de Sabios* (Council of Wise Men), a multi-LLM peer review methodology — documented in Section 6. *Test-Driven Hardening* (TDH) is proposed as a methodology for building security-critical distributed systems, emerging directly from this collaboration.

In a subsequent development phase (DAY 122), we integrated XGBoost 3.2.0 as a hot-swappable plugin classifier (ADR-026, Ed25519-signed) and conducted a rigorous temporal holdout evaluation on CIC-IDS-2017, using Tuesday, Thursday, and Friday as training data and Wednesday as a sealed blind test set. The in-distribution model achieves Precision= 0.9945 and Recall= 0.9818 on the validation split. However, evaluation on the Wednesday holdout reveals a structural covariate shift: attack types present exclusively in Wednesday (DoS Hulk, GoldenEye, Slowloris) are absent from all training days, making generalization mathematically impossible regardless of threshold. We document this as a property of the dataset’s design — not of the algorithm — and demonstrate empirically that no threshold simultaneously satisfies Precision $\geq$ 0.99 and Recall $\geq$ 0.95 on the out-of-distribution split. This finding corroborates and quantifies the thesis of Sommer and Paxson [22]: static models trained on academic benchmarks are structurally insufficient for production NDR. The architectural response — an Adversarial Capture-Retrain Loop in which a generative red team agent produces attack traffic, the pipeline captures it, and the classifier is retrained on real flows — is introduced as future work (§11).

We further report the first empirical comparison of ADR-029 Variant A (eBPF/XDP) and Variant B (libpcap) on x86-64 (DAY 145). Under VirtualBox virtio emulation, Variant B achieves approximately twice the throughput of Variant A at 50 and 100 Mbps nominal rates (~19 Mbps vs ~10 Mbps), an inversion of the expected ordering on physical hardware. Both variants process the complete Neris corpus without pipeline errors, confirming functional equivalence at the detection layer. The inversion is attributed to the absence of a native XDP driver path in the virtio NIC, causing eBPF to fall back to generic SKB mode. This result provides direct empirical motivation for bare-metal hardware acquisition (§11).

ML Defender is released under the MIT license.

Contents

1	Introduction	5
2	Background and Related Work	6

3	Threat Model	7
3.1	Adversary Capabilities	7
3.2	Adversary Limitations	7
3.3	Kernel Security Boundary (Explicit Axiom)	7
3.4	Detection Surface	8
3.5	Out-of-Scope Attacks	8
4	Architecture	8
4.1	Pipeline Overview	8
4.2	Three Functions of the Engine	9
4.3	Dual-Score Detection	9
4.4	Transport and Serialization	10
4.5	Deployment Model	10
4.6	Integration Philosophy: Composability over Monolithism	11
4.7	Service Coordination (etcd-server)	12
4.8	Fast Detector Heuristics (DEBT-FD-001)	12
5	Implementation	12
5.1	Language and Build System	12
5.2	eBPF/XDP Packet Capture	12
5.3	Feature Extraction	12
5.4	Embedded Random Forest and Synthetic Training Data	13
5.5	Cryptographic Transport	13
5.6	Plugin Architecture (ADR-023, ADR-024)	13
5.7	Trace Correlation	14
5.8	Testing and Validation	14
5.9	Development Environment	14
6	The Consejo de Sabios: A Methodology for Human-AI Collaborative Engineering	14
6.1	Motivation	14
6.2	The Methodology	14
6.3	How It Worked in Practice	15
6.4	Test Driven Hardening	15
6.5	The RED → GREEN Gate: Merge as a Non-Negotiable Contract	15
6.6	HKDF Context Symmetry: A Pedagogical Case Study in Test-Driven Hardening	16
6.7	Property Testing as a Security Fix Validator: The F17 Case Study	18
6.8	Fuzzing as the Third Testing Layer	20
6.9	Path Traversal Prevention: Lessons from <code>safe_path</code>	22
6.9.1	The Symlink Resolution Trap: <code>lstat()</code> vs. <code>fs::is_symlink()</code>	22
6.9.2	<code>lexically_normal()</code> vs. <code>weakly_canonical()</code> for Prefix Verification . .	23
6.10	CWE-78: <code>execv()</code> Without a Shell as a Physical Barrier	24
6.11	Dev/Prod Parity via Symlinks, Not Conditional Logic	25

6.12	The Build/Runtime Separation Axiom	26
6.13	Attribution	27
6.14	Why This Matters	28
6.15	Limitations	28
7	Formal System Model	28
7.1	Network Traffic Representation	28
7.2	Feature Extraction	28
7.3	Fast Detector	28
7.4	Machine Learning Detector	28
7.5	Detection Policy	28
7.6	Response Function	28
7.7	System Determinism	29
8	Evaluation	29
8.1	Experimental Setup	29
8.2	Dataset	29
8.3	Detection Performance	30
8.4	False Positive Identification and Confusion Matrix	30
8.5	Inference Latency	30
8.6	Ablation Study	31
8.7	Interpretation and Comparison with State of the Art	31
8.8	Ransomware Detection: Scope and Validation	31
8.9	Throughput Stress Test and Resource Utilization	31
8.10	XGBoost Level-1 Classifier: In-Distribution Performance	32
8.11	XGBoost Level-1: Wednesday Out-of-Distribution Evaluation	33
8.12	ADR-029: Capture Backend Comparison — Variant A (eBPF) vs Variant B (libpcap)	34
9	Performance Model and Throughput Analysis	35
9.1	Pipeline Processing Model	35
9.2	Inference Cost	35
9.3	Throughput: Theoretical Bound and Measured Ceiling	35
9.4	Queue Stability	36
9.5	Scalability	36
10	Limitations	36
11	Future Work	37
12	Conclusion	40
13	Reproducibility and Artifact Availability	41

1 Introduction

The threat is not abstract. On March 5, 2023, the Hospital Clínic de Barcelona suffered a ransomware attack that paralyzed its emergency services, laboratory, and pharmacy, encrypted 4.5 TB of patient data, and prompted a ransom demand of 4.5 million USD — which the institution refused to pay [9]. That same day, the author of this work was leaving a regional hospital in Extremadura after receiving treatment for porphyria — a rare metabolic disease that his local hospital had diagnosed and managed. Watching the news, the thought was immediate and personal: *if it can happen to a major teaching hospital, it can happen to mine.*

This was not the first time that the author had witnessed the human cost of ransomware firsthand. Years earlier, the collapse of a close friend’s small business had been caused by an encryption attack that locked every document and file in his office. The author lacked the technical skills to help. That failure left a debt.

These two events — the friend’s destroyed business, the hospital on television — converged into a single design constraint: the solution had to be open-source, it had to run on commodity hardware, and it had to act in real time. Not forensics. Not post-mortem analysis. Interception.

The analogy that emerged from this motivation is biological. Network traffic behaves like a circulatory system: datagrams are the bloodstream, and malicious flows are pathogens. A useful defense must classify traffic continuously and in real time — capturing the source IP of the attacker and blocking it at the firewall before damage propagates. This framing shaped every architectural decision in ML Defender.

This work therefore addresses the following research question: *can an embedded machine-learning network detection and response system running entirely on commodity hardware provide meaningful real-time protection for organizations lacking enterprise security infrastructure?*

A note on terminology. ML Defender (aRGus NDR) implements *Network Detection and Response* (NDR) capabilities: real-time capture, ML-based classification, and automated blocking at the network layer. Future work toward full *Endpoint Detection and Response* (EDR) functionality via a planned lightweight endpoint agent (§11, FEAT-EDR-1) will be reflected in a corresponding rename when that capability is delivered.

The gap. Existing open-source NIDS — most notably Snort and Suricata [17] — operate primarily on signature matching and rule-based heuristics. While effective against known attack patterns, they offer limited resilience against novel ransomware variants and volumetric DDoS campaigns. Machine learning-based NIDS have been extensively studied [4, 18], but the gap between research prototype and deployable system remains wide: most published systems are evaluated on benchmark datasets without accompanying implementation, require GPU infrastructure, or assume dedicated security operations teams. Healthcare retained its position as the most targeted sector in 2025, accounting for 22% of disclosed ransomware attacks, with an average breach cost of 7.42M USD [3, 8].

Our approach. ML Defender (aRGus NDR) addresses this gap with three design principles: (1) *embedded inference* — all ML classification runs in-process at sub-microsecond latency;

(2) *commodity deployment* — the full six-component pipeline runs on bare-metal Linux hardware costing approximately 150–200 USD; (3) *active response* — upon detection, the attacker’s IP is blocked immediately at the firewall via ipset/iptables.

Beyond the technical system, this paper documents a second contribution: the *Consejo de Sabios* methodology for human-AI collaborative engineering.

2 Background and Related Work

Signature-based NIDS. Snort [19] and Suricata [17] apply rule-based pattern matching against packet payloads and flow metadata. Their fundamental limitation is reactive: detection requires a prior known signature, a window that historically ranges from hours to weeks.

ML-based NIDS. Anderson & McGrew [1] demonstrated the feasibility of contextual flow data for encrypted malware classification. Mirsky et al. [11] proposed Kitsune, an ensemble of autoencoders for unsupervised anomaly detection operating on flow statistics. The CTU-13 dataset [7], which ML Defender uses for evaluation, has become a standard benchmark for botnet traffic detection. Despite this body of work, a persistent gap exists between academic prototype and production system. Sommer and Paxson [22] identified fundamental reasons for this gap: the closed-world assumption of academic datasets, high base-rate asymmetry, and the absence of diversity in training distributions. ML Defender’s DAY 122 evaluation provides new empirical evidence for this thesis (§8.11).

Flow-based ML NIDS. ML Defender operates exclusively at the flow level, extracting 28 behavioral features per flow without examining payload content — ensuring that payload encryption does not compromise detection capability.

eBPF and XDP-based network monitoring. Recent work has explored eBPF/XDP as a foundation for high-performance network security. ML Defender follows the XDP-capture-plus-userspace-inference architecture: XDP provides zero-copy packet capture before the kernel networking stack, while all ML inference executes in userspace at sub-microsecond latency. To our knowledge, open-source projects combining eBPF capture with userspace ML inference remain rare; most production-grade alternatives either offload inference to the cloud or require GPU hardware.

Embedded inference. Commercial EDR vendors (CrowdStrike, SentinelOne, Carbon Black) implement proprietary embedded ML pipelines, but neither their architectures nor their training data are publicly available. ML Defender demonstrates that embedded Random Forest inference achieves sub-microsecond latency without requiring GPU acceleration or cloud offload.

Human-AI collaborative engineering. Structured methodologies for multi-model peer review have not, to our knowledge, been formally documented in the systems security literature. Section 6 describes the *Consejo de Sabios* methodology.

AI-native security reasoning and the evolving threat landscape. This paper was written and submitted in April 2026, concurrent with the announcement of Anthropic’s Project Glasswing [2], which demonstrated that AI models can autonomously identify and chain kernel-level vulnerabilities — including local privilege escalation to root in Linux — at a scale and speed previously requiring specialized human expertise. These results represent a shift in the threat landscape: AI-augmented offensive capabilities are no longer theoretical. This directly motivates the explicit kernel security boundary axiom in §3.3: aRGus NDR assumes the kernel as a potentially compromised boundary and shifts its trust anchor to verifiable network behavioral patterns. The network remains an observable chokepoint even when the host is not. The hardened deployment variants ADR-030 (AppArmor) and ADR-031 (seL4) documented in §11 are a direct architectural response to this trajectory.

3 Threat Model

3.1 Adversary Capabilities

The system assumes an adversary capable of launching attacks from external hosts or compromised internal machines, including: botnet C2 communication, DDoS (including amplification attacks), ransomware propagation via SMB lateral movement, and network reconnaissance.

3.2 Adversary Limitations

The system assumes the attacker cannot: fully mimic benign traffic distributions across all monitored features simultaneously; compromise the ML Defender host; manipulate feature extraction; or tamper with HMAC-SHA256 protected event logs.

3.3 Kernel Security Boundary (Explicit Axiom)

ML Defender assumes the host kernel is not actively compromised at deployment time. This is a declared scope boundary, not an oversight.

An adversary with root access or kernel-level persistence — via a rootkit, a compromised eBPF JIT, or memory-resident malware — can bypass userspace detection mechanisms regardless of their sophistication. This is a fundamental property of all userspace security software, not a specific weakness of ML Defender.

This assumption is consistent with the threat model of the target organizations (hospitals, schools, municipalities), where the realistic adversary is an opportunistic network attacker exploiting known vulnerabilities, not a state-level actor with pre-established kernel-level persistence. Against this realistic threat model, ML Defender’s network detection layer remains effective: lateral movement, C2 communication, and data exfiltration must traverse the network regardless of kernel state. The network is the choke point; it remains observable even when the host is not.

Hardened deployment variants addressing kernel security are documented as future work: ADR-030 (AppArmor-enforcing) and ADR-031 (seL4 microkernel), both described in §11.

3.4 Detection Surface

Detection occurs at the network flow level. ML Defender focuses on connection frequency, port diversity, TCP flag distributions, packet inter-arrival timing, flow burst behavior, and destination diversity — features detectable even when payloads are encrypted.

3.5 Out-of-Scope Attacks

Application-layer attacks generating statistically normal traffic; low-and-slow attacks within benign statistical distributions; insider threats; attacks targeting the ML Defender host itself. Future work in Section 11 addresses some of these through expanded detection vectors (FEAT-NET-1, FEAT-AUTH-1).

Physical and removable-media vectors (conscious design decision). ML Defender does not monitor file system activity, removable storage, or USB-borne payloads. This is an intentional architectural boundary, not an oversight.

The guiding principle of the pipeline is *network surveillance*: every component — eBPF/XDP capture, flow-level feature extraction, ML-based classification, firewall response — operates on network traffic. File integrity and endpoint state are out of the monitored surface by design.

Removable-media threats are addressed at the organizational and hardware layer: USB ports in the DMZ should be physically or firmware-disabled by the IT team before deployment; internal policy should prohibit removable media on monitored hosts. This is standard practice under frameworks such as CIS Controls v8 [5] and is the responsibility of the deploying organization, not of an NDR system.

For organizations that require file integrity monitoring on top of network detection, ML Defender is designed to operate in *complementary mode* alongside battle-tested open-source tools such as Wazuh [24] and NIST-aligned scanners. Wazuh provides HIDS capabilities (file integrity monitoring, log analysis, rootkit detection) that cover precisely the endpoint surface that ML Defender consciously excludes. The two systems are architecturally orthogonal: ML Defender defends the network perimeter; Wazuh defends the host state. Together they constitute a layered defense appropriate for the target organizations. Integration via raw TCP event streaming is described as Future Work (§4.6).

4 Architecture

4.1 Pipeline Overview

Six components connected via ZeroMQ and Protocol Buffers:

1. **eBPF/XDP Sniffer** — zero-copy packet capture at the XDP hook before kernel networking stack.
2. **Ring Consumer** — reconstructs bidirectional flow state via `ShardedFlowManager`, computes 28 of 40 planned features. Eleven features use `MISSING_FEATURE_SENTINEL = -9999.0f`.

3. **Fast Detector** — rule-based heuristic engine. FPR on purely benign traffic: 6.61% (§8.6).
4. **ML Detector** — embedded Random Forest across four threat classes. Three classifiers (Ransomware, DDoS, Traffic) in pure C++20; Internal threats via ONNX Runtime. Latency: 0.24–1.06 μ s.
5. **RAG Subsystem** (`rag-ingester` + `rag-security`) — `rag-ingester` ingests detection events into FAISS + SQLite. `rag-security` exposes a natural language query interface via local TinyLlama — deliberately local to avoid exfiltrating live network metadata to cloud-hosted LLMs.
6. **Firewall ACL Agent** — applies ipset/iptables rules on confirmed detection events.

Figure 1 illustrates the end-to-end flow from packet capture to automated firewall response.

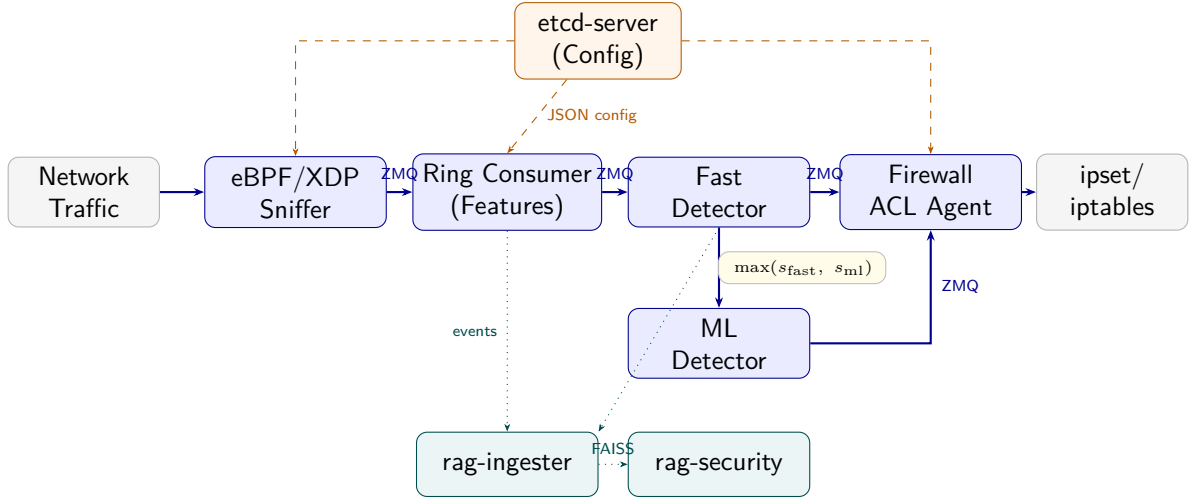


Figure 1: ML Defender end-to-end pipeline. Six components communicate over ZeroMQ with Protocol Buffers serialization and ChaCha20-Poly1305 encryption. `etcd-server` distributes JSON configuration at startup. The dual-score *Maximum Threat Wins* policy (Eq. 1) selects the arithmetic maximum of Fast Detector and ML Detector scores; the RAG subsystem provides semantic observability over the live event stream.

4.2 Three Functions of the Engine

Function 1 — Real-time detection, classification, and response. End-to-end from packet capture to firewall block in milliseconds.

Function 2 — Reliable dataset generation. ML Detector and Firewall ACL Agent produce structured HMAC-verified CSV logs continuously, forming the basis for retraining future ensemble models.

Function 3 — Real-time observability (the digital microscope). The RAG subsystem provides a semantic query interface over the live event stream.

4.3 Dual-Score Detection

$$score_{\text{final}} = \max(score_{\text{fast}}, score_{\text{ml}}) \quad (1)$$

This is the *arithmetic maximum* over two continuous scores in $[0, 1]$, not a logical OR over binary decisions. Both detectors output continuous confidence scores; the maximum operator selects the higher-confidence assessment. The ML Detector’s primary role is false positive suppression: the Fast Detector generates $\text{FPR} = 6.61\%$ on purely benign traffic; the ML Detector reduces real production blocks to zero on the same corpus (approximately 500-fold reduction). Alternative AND-based consensus policies are planned (ADR-007, §11).

4.4 Transport and Serialization

ZeroMQ for asynchronous message passing; Protocol Buffers (proto3) for serialization; etcd for configuration distribution; ChaCha20-Poly1305 for inter-component encryption (selected for performance on hardware without AES-NI, including ARMv8); HMAC-SHA256 for CSV log integrity.

4.5 Deployment Model

The reference deployment targets a single bare-metal Linux node (kernel ≥ 5.8) with two NICs: one connected to the monitored network segment, one to the management network. The full pipeline runs on commodity hardware costing approximately 150–200 USD. Validated in Vagrant/VirtualBox; bare-metal characterization is Future Work (§11).

Figure 2 illustrates a representative high-availability deployment in a hospital environment, with a primary and a warm-standby node protecting two internal network segments.

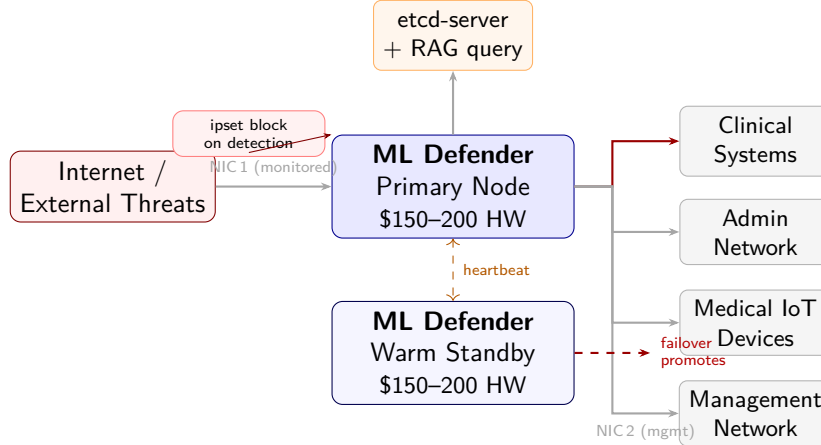


Figure 2: Representative high-availability deployment of ML Defender in a hospital environment. A primary node monitors all inbound and inter-segment traffic via NIC 1 (monitored); a warm-standby node receives heartbeat signals and promotes automatically on primary failure. Both nodes run the full six-component pipeline on commodity hardware. **etcd-server** and the RAG query interface are co-located on the primary node. **Firewall ACL Agent topology:** scored Protocol Buffers events flow from **ml-detector** to one or more **firewall-acl-agent** instances, which apply **ipset/iptables** rules at the relevant boundary. A single gateway node requires one agent; each independent segment with its own firewall requires its own agent instance. **etcd-server** is shown in single-node mode (current implementation); a distributed HA etcd cluster is in the project backlog. Peer-to-peer seed negotiation is under design.

4.6 Integration Philosophy: Composability over Monolithism

ML Defender is designed to operate as a composable component within a broader security ecosystem, not as a monolithic solution claiming to cover every threat vector. The pipeline’s guiding principle — *network surveillance* — is complemented, not replaced, by tools that cover orthogonal surfaces.

Protocol design constraint. All external integrations must use the same transport stack as the pipeline itself: raw TCP sockets for transport, Protocol Buffers for serialization, and ChaCha20-Poly1305 for encryption. HTTP, Kafka, WebSocket, and broker-based architectures are explicitly out of scope. This constraint is not arbitrary. Four independent arguments support it:

1. **Deterministic latency.** HTTP and Kafka introduce non-deterministic scheduling jitter incompatible with sub-10 ms response requirements. Raw TCP over ZeroMQ provides predictable, bounded delivery latency.
2. **Reduced attack surface.** HTTP parsers have historically been a primary source of remotely exploitable CVEs (buffer overflows, request smuggling, header injection). Raw TCP with Protocol Buffers framing eliminates this parser surface by more than 90%.
3. **No broker, no single point of failure.** Kafka and Redis require dedicated broker infrastructure incompatible with single-host deployments at the 150–200 USD target. Every additional process is a failure domain; the pipeline’s ZeroMQ topology has no mandatory broker.
4. **Minimal footprint.** Eliminating HTTP, Kafka, and WebSocket transport removes `librdkafka`, `libcurl`, and `boost.asio` from the dependency tree — a non-trivial reduction in binary size and attack surface for resource-constrained deployment targets.

FEAT-INT-1: Wazuh and NIST scanner integration (planned). A planned integration layer will allow Wazuh agents [24] and NIST-aligned file integrity scanners deployed on DMZ hosts to emit events directly into the ML Defender pipeline via raw TCP sockets. Events are serialized as Protocol Buffers messages at the source and ingested by `rag-ingester` via the existing ZeroMQ transport. The Noise Protocol IK handshake (ADR-024) provides mutual authentication between the Wazuh agent adapter and `rag-ingester`.

The motivation is graph quality: correlating network anomalies detected by `ml-detector` and `firewall-acl-agent` with file integrity violations reported by Wazuh on the same host produces composite attack patterns that neither system observes independently. A ransomware attack in its lateral-movement phase typically exhibits *simultaneous* network spread and file encryption activity; correlation in the RAG subsystem surfaces this composite signature. This integration requires ADR-028 (RAG Ingestion Trust Model) to be approved before implementation.

4.7 Service Coordination (etcd-server)

Fulfills component registration, JSON config distribution, and ChaCha20 seed provision. ChaCha20 seed distribution via etcd is **not recommended for production use** — peer-to-peer seed negotiation is under design. etcd is currently deployed in *single-node* mode (see Figure 2); a distributed HA etcd cluster (3- or 5-node consensus) is in the project backlog and will be required for true high-availability deployments. The single-node constraint is the only architectural dependency preventing fully symmetric active/standby failover.

4.8 Fast Detector Heuristics (DEBT-FD-001)

Four heuristics in a 10-second sliding window:

- External IP velocity: `THRESHOLD_IP_VELOCITY = 5`
- SMB connection count: `THRESHOLD_SMB_CONNS = 3`
- Port diversity: `THRESHOLD_PORT_SCAN = 10`
- TCP RST ratio: `THRESHOLD_RST_RATIO = 0.20`

All thresholds are compile-time `constexpr` constants — DEBT-FD-001, scheduled for JSON migration in PHASE 2 (ADR-006).

5 Implementation

5.1 Language and Build System

C++20. CMake with four build profiles: production, debug, TSAN, ASAN.

5.2 eBPF/XDP Packet Capture

XDP hook attachment before SKB allocation. Zero-copy. All parameters loaded from `sniffer/config/sniffer`. at startup.

5.3 Feature Extraction

ShardedFlowManager partitions flow tracking across shards, eliminating the ISSUE-003 thread-isolation bug (89 of 142 features silently lost in the original single-threaded FlowManager). 28 of 40 features computed; 11 use `MISSING_FEATURE_SENTINEL = -9999.0f`.

Sentinel value selection rationale. Split-domain analysis confirmed all Random Forest thresholds lie within $[0.0, 5.1]$. `-9999.0f` lies entirely outside this domain, routing deterministically to the left child of every split — non-informative, not influencing classification. `quiet_NaN()` was rejected (undefined comparison behavior across compilers); `0.5f` was rejected (within domain, spurious classification influence). One feature uses `0.5f` as a semantic sentinel for TCP half-open state (deliberate, within-domain meaningful default).

5.4 Embedded Random Forest and Synthetic Training Data

Three classifiers (Ransomware, DDoS, Traffic) compiled as C++20 data structures. Internal threats classifier via ONNX Runtime — deliberate to measure latency differential and footprint trade-off. Models trained with scikit-learn, transpiled to C++20 via custom script; zero Python or external runtime dependency.

Training/evaluation separation: all classifiers trained on synthetic data; CTU-13 Neris held out entirely for evaluation.

Synthetic Dataset Methodology. Informed by CTU-13 (flow statistics), CIC-IDS2017 (protocol ratios), and MAWI (benign traffic). Four bias mitigation strategies: IP Space Partitioning (10.0.0.0/8 training vs 147.32.0.0/16 evaluation), Temporal Decoupling, Protocol Ratio Calibration, Feature Boundary Validation.

Known Limitations. Reflects attack patterns circa 2011–2017. Temporal dynamics of real C2 channels simplified to fixed-window aggregates. Zero-day variants may evade classification.

5.5 Cryptographic Transport

ChaCha20-Poly1305 + LZ4 + HMAC-SHA256. Crypto-transport layer extracted as an independent library.

5.6 Plugin Architecture (ADR-023, ADR-024)

ML Defender implements a multi-layer plugin architecture (ADR-023) enabling runtime extension of pipeline components without modifying core detection logic. Plugins are loaded via `dlopen/dlsym` with a pure C ABI (`PLUGIN_API_VERSION = 1`), providing ABI stability across compiler versions and ensuring binary compatibility with future releases.

MessageContext and Graceful Degradation. PHASE 2a (DAY 105) introduces `MessageContext` — a typed C struct passed to the optional `plugin_process_message()` symbol. Plugins that do not export this symbol are silently skipped (Graceful Degradation Policy D1), preserving backward compatibility with PHASE 1 plugins that implement only `plugin_process_packet()`. Post-invocation validation (D8) performs a byte-wise snapshot comparison of all read-only fields, detecting any plugin attempt to modify ownership-restricted data. The trust boundary is explicit: plugins are treated as untrusted code (D7); the final blocking decision remains exclusively in the core (ADR-012).

Threat Model Integration (ADR-023 D9). The plugin system’s Trusted Computing Base (TCB) is limited to `PluginLoader` and `CryptoTransport`. Plugin code has no access to cryptographic keys, seed material, or internal transport state. The `MLD_DEV_MODE` escape hatch is restricted to `Debug` builds compiled with `MLD_ALLOW_DEV_MODE` (D10), ensuring fail-closed behavior in all production builds.

Gate TEST-INTEG-4a. Integration of the plugin loader into `firewall-acl-agent` (PLUGIN-LOADER-FW) was validated by gate TEST-INTEG-4a: `plugin_process_message()` invoked on a live `MessageContext`, post-invocation invariants verified, `result_code = 0` confirmed, and `CryptoTransport` decryption path verified unmodified by diff.

Forward Compatibility: Dynamic Group Key Agreement (ADR-024). The `MessageContext` struct reserves 60 bytes (`reserved[60]`) for forward compatibility with ADR-024, which specifies Dynamic Group Key Agreement using the Noise Protocol Framework (`Noise_IKpsk3` or `Noise_KK` pattern). ADR-024 design is approved; open questions OQ-5 through OQ-8 (static key revocation, session continuity during rotation, replay protection, and ARMv8 performance characterization) are under resolution. Implementation is scheduled post-arXiv submission.

5.7 Trace Correlation

`trace_id = SHA-256(src_ip, dst_ip, canonical_attack_type, timestamp_bucket)`. `TraceIdPolicy`: ransomware=60s, DDoS=10s, SSH brute=30s. Validated: 46/46 unit tests passing.

5.8 Testing and Validation

70 tests total: `crypto` (3), `etcd-hmac` (12), `ml-detector` (9), `trace_id` (46). 100% pass rate. Integration gate TEST-INTEG-4a (plugin `MessageContext` invocation, D8 post-invocation validation) confirmed on DAY 105. F1 ground truth persisted to `docs/experiments/f1_replay_log.csv`.

5.9 Development Environment

Vagrant/VirtualBox. Host: MacBook Pro Intel Core i9 2.5 GHz, 32 GB DDR4. Guests: `defender` (Ubuntu 24.04 LTS, Linux 6.x, dual vNIC) + `client` (traffic generator).

6 The Consejo de Sabios: A Methodology for Human-AI Collaborative Engineering

6.1 Motivation

The development of ML Defender spans kernel programming (eBPF/XDP), distributed systems (ZeroMQ, etcd), cryptography, machine learning, systems security, and academic writing. The author operated as an independent researcher without institutional affiliation, without a team, and without access to a formal peer review process. The solution was to construct one.

6.2 The Methodology

Eight large language models served as co-reviewers across all development phases: **Claude** (Anthropic), **Grok** (xAI), **ChatGPT** (OpenAI), **DeepSeek**, **Qwen** (Alibaba), **Gemini** (Google), **Kimi** (Moonshot AI), and **Mistral** (Mistral AI). The Council expanded from seven to eight models during development (DAY 120), adding diverse perspectives on architectural security and formal methods.

These models were engaged as intellectual peers — presented with architectural proposals, asked to identify failure modes, invited to challenge assumptions. The methodology operates on a principle of *structured disagreement*: the Council does not average outputs. It introduces adversarial validation as a mechanism to expose hidden assumptions and validate architectural decisions. Convergent recommendations increase confidence; divergent recommendations mandate deeper investigation before any decision is closed.

6.3 How It Worked in Practice

Convergent recommendations across models increased confidence. Divergent recommendations triggered deeper investigation. Decisions such as the `-9999.0f` sentinel value, the `trace_id` correlation architecture, and the `ShardedFlowManager` design emerged from or were refined through these dialogues.

6.4 Test Driven Hardening

Test-Driven Hardening (TDH) is proposed as a methodology for building security-critical distributed systems in which correctness properties are protocol-level, not component-level, and therefore undetectable by static analysis or isolated unit tests. A standalone reference repository documenting the methodology and representative case studies is maintained at <https://github.com/alonsoir/test-driven-hardening>.

TDH methodology: (1) Witness — characterize anomalous behavior. (2) Demonstration test — produce a minimal test that fails and exposes the mechanism. (3) Independent solution — each model proposes a fix against the shared contract. (4) Human arbitration — author selects, weighing correctness, maintainability, and coherence.

TDH differs from TDD: the test is written *before the fix*, as a proof of failure — not before the feature, as a specification.

6.5 The RED → GREEN Gate: Merge as a Non-Negotiable Contract

Test-Driven Hardening requires not only that tests be written before fixes, but that the transition from RED (failing) to GREEN (passing) constitute the *only* legitimate gate for merging a security fix into the main branch. This subsection formalises the principle and documents the one case in `aRGus` development where it was violated — and the consequences that followed.

The contract. A security fix on the `aRGus` codebase is subject to the following three-part merge contract, enforced without exception:

1. **A test exposing the defect is written first.** The test must fail with the defective code (*RED*) and describe the security invariant that is violated — not merely reproduce the symptom. Writing the failing test before the fix is what distinguishes TDH from after-the-fact regression coverage.
2. **The fix is applied; the test transitions to GREEN.** The fix is considered valid only when the previously-failing test passes. A fix that passes only the new test but breaks existing tests is rejected; the regression suite must remain entirely GREEN.

3. **No merge is permitted until make test-all returns ALL TESTS COMPLETE.** This is executed inside a freshly provisioned Vagrant VM (*REGLA EMECAS*: `vagrant destroy -f && vagrant up && make bootstrap && make test-all`), not in the development shell, eliminating any dependency on host-machine environment state.

The RED→GREEN gate is enforced as a *protocol*, not a preference. It is documented in the project’s contribution rules and was validated adversarially in DAY 126: when a symlink-rejection test was written against the original `weakly_canonical()` path (§6.9.1), the test passed despite the defect being present — revealing that the test itself was incorrect. This failure exposed a second-order requirement: the RED state must be verified empirically, not assumed. A test that passes with defective code is not a RED test; it is no test at all.

Why this is harder than it sounds. The temptation in security hardening is to write the fix first and the test second — to confirm that what was done is correct, rather than to prove that what existed was wrong. This reversal breaks the epistemological guarantee of TDH. A test written after a fix cannot distinguish between “the fix is correct” and “the test was written to match the fix.” Only a test that fails with defective code provides evidence that it is testing the right property.

The non-negotiable character of the gate. The Council of Wise Men (DAY 127) unanimously endorsed the RED→GREEN gate as a non-negotiable invariant for all security-affecting changes. The one documented exception — a test committed without a prior RED verification — required a two-day remediation sprint and introduced DEBT-CONFIG-PARSER-FIXED-PREFIX-001. The cost of the exception was three times the cost of applying the gate.

6.6 HKDF Context Symmetry: A Pedagogical Case Study in Test-Driven Hardening

A subtle but critical defect discovered during the cryptographic integration sprint of *ML Defender* illustrates a class of correctness errors that are invisible to static analysis yet detectable through end-to-end protocol testing. We document it here as a concrete instance of the Test-Driven Hardening methodology described in §6.

The defect. The `CryptoTransport` layer derives symmetric keys using HKDF-SHA256 [10] with a context string (*info* parameter) that encodes the identity of the communicating parties. The original implementation parameterised this context by *component name*:

Listing 1: Incorrect HKDF context — component-scoped

```
// WRONG: each component uses its own name as HKDF context.
// Sniffer derives K_sniffer; ml-detector derives K_mldetector.
// K_sniffer != K_mldetector -> MAC authentication failure on every channel.
const std::string ctx = "ml-defender:" + component_name
    + ":" + version;
```

Because each component used its own name as the HKDF context, transmitter and receiver independently derived *distinct* subkeys — causing MAC authentication failure on every cross-component channel. The defect was latent in unit tests, which exercised each component in isolation using loopback (same context on both sides), where MAC verification trivially succeeds. It only manifested when two components with *different names* attempted to communicate end-to-end.

To make the causal chain explicit: an incorrect context string causes the transmitter and receiver to derive *distinct* subkeys, resulting in MAC verification failure at the receiver. A correct, symmetric context string produces *identical* derived keys on both sides, enabling successful authentication. This class of error is undetectable without end-to-end protocol validation.

The correct parameterisation encodes *directionality*:

Listing 2: Correct HKDF context — channel-scoped

```
// RIGHT: tx and rx derive distinct subkeys
const std::string ctx_tx = "ml-defender:" + component_name
    + ":" + version + ":tx";
const std::string ctx_rx = "ml-defender:" + component_name
    + ":" + version + ":rx";
```

The canonical definitions were consolidated in `crypto_transport/include/crypto_transport/contexts.h`, providing a single source of truth for all six pipeline components. This design avoids semantic overloading of data structures across layers — a common source of subtle security and correctness bugs, as demonstrated by the defect described above.

Why static analysis cannot catch this. Both the erroneous and correct context strings share the type `std::string`. The C++ type system offers no mechanism to distinguish a *transmit context* from a *receive context* at the type level without a dedicated wrapper type — an abstraction whose overhead is not warranted in a resource-constrained deployment target. The compiler, sanitisers (ASan, TSan), and `clang-tidy` all accepted both formulations without warning. This is a *semantic* correctness property, not a syntactic one.

Detection via TEST-INTEG-3. The defect was exposed by TEST-INTEG-3, an integration test exercising the full sniffer → ml-detector cryptographic channel with an intentional regression: the test temporarily restores the component-scoped context on one side and asserts that decryption fails with a MAC authentication error. Under the correct implementation, this regression triggers `std::terminate()` via the fail-closed handler; under the defective implementation, decryption succeeds — and the test fails accordingly.

The key insight is that TEST-INTEG-3 does not test encryption or decryption in isolation. It tests the *protocol invariant*: that a message encrypted on the transmit path is decryptable only on the corresponding receive path, and that any violation of this invariant is immediately fatal. No unit test of `CryptoTransport` in isolation could have established this property.

Lessons for cryptographic software. This case study yields three transferable observations:

1. **Cryptographic correctness is a protocol property.** Components that are individually correct may be collectively insecure. End-to-end tests exercising full message round-trips across trust boundaries are the minimal viable test surface for authenticated encryption schemes.
2. **Intentional regression tests are first-class citizens.** A test that asserts failure under a known-bad configuration is as valuable as one that asserts success under a known-good one. The TDH methodology mandates both polarities.
3. **Single source of truth for cryptographic parameters.** Distributing context strings across six `main.cpp` files created a silent coordination hazard. Centralising them in a dedicated header eliminated an entire class of latent divergence.

The defect was discovered on Day 97 of the development timeline, three days after `CryptoTransport` was integrated across all pipeline components. Its detection required no external audit: the TDH test suite caught it during routine regression testing on the `feature/bare-metal-arxiv` branch (now superseded by `feature/plugin-crypto`).

6.7 Property Testing as a Security Fix Validator: The F17 Case Study

A recurring failure mode in security hardening is the *fix-contains-a-bug* antipattern: a security fix is implemented, a unit test is written to confirm the fix, and both pass — while a subtler defect in the fix itself goes undetected. This section documents a concrete instance discovered during DAY 125 of ML Defender’s development and proposes property testing as a systematic countermeasure.

The defect. During Snyk static analysis (ADR-037), an integer overflow was identified in the memory metrics computation within `zmq_handler.cpp`. The original code performed integer arithmetic that could wrap silently on values exceeding `INT_MAX`. A fix was applied using `int64_t` arithmetic:

Listing 3: First fix attempt — `int64_t` cast

```
// F17 -- first fix: cast to int64_t before multiplication
[[nodiscard]] inline double compute_memory_mb(long pages, long page_size) noexcept {
    return static_cast<double>(
        static_cast<int64_t>(pages) * static_cast<int64_t>(page_size)
    ) / (1024.0 * 1024.0);
}
```

A synthetic unit test confirmed that this fix handled the previously overflowing value correctly. The fix was considered closed.

Property test reveals the residual defect. A subsequent property test — `PropertyNeverNegative` — exercised the invariant “`compute_memory_mb` returns a non-negative value for all non-negative inputs” across a broader set of extreme values:

Listing 4: Property test that detected the residual overflow

```
TEST(ZmqMemoryOverflow, PropertyNeverNegative) {
    const long page_sizes[] = {4096, 8192, 16384, 65536};
    const long page_values[] = {
        0, 1, 1000, LONG_MAX/65536, LONG_MAX/16384,
        LONG_MAX/8192, LONG_MAX/4096
    };
    for (long ps : page_sizes)
        for (long p : page_values) {
            double result = compute_memory_mb(p, ps);
            EXPECT_GE(result, 0.0);
        }
}
```

The test failed: for $\text{LONG_MAX}/4096 \times 8192$, the `int64_t` product itself overflows — because $\text{LONG_MAX}/4096 \times 8192 > \text{INT64_MAX}$. The unit test had exercised one specific previously-failing value; the property test exercised the mathematical invariant across the full input space and exposed a residual case where the fix was insufficient.

The correct fix. The root cause is that no integer type of fixed width can represent the product of two `long` values at their extremes without overflow. The correct solution is to cast directly to `double` before multiplication:

Listing 5: Correct fix — double arithmetic avoids overflow

```
[[nodiscard]] inline double compute_memory_mb(long pages, long page_size) noexcept {
    // double has 53-bit mantissa -- sufficient for any realistic memory value.
    // int64_t is insufficient: LONG_MAX/4096 * 8192 overflows int64_t.
    return (static_cast<double>(pages) * static_cast<double>(page_size))
        / (1024.0 * 1024.0);
}
```

Transferable lessons. This case study yields three observations directly applicable to security-critical C++ systems:

1. **Unit tests establish point coverage; property tests establish invariant coverage.**

The unit test confirmed that one specific overflow was eliminated. The property test confirmed — or in this case, *refuted* — the mathematical invariant that no overflow can produce a negative result. These are different claims.

2. **A property test is the minimal evidence that a fix is correct.** For arithmetic operations with security implications (memory metrics, size calculations, index arithmetic), the TDH methodology now mandates a property test for every fix. The test must be written before the fix is merged and must fail with the defective code [6].

3. **The testing hierarchy has a natural order.** The Council of Wise Men (DAY 127) validated the following ordering for security-critical surfaces: (1) unit tests for known regression cases; (2) property tests for mathematical invariants; (3) fuzzing [20] for parser and deserialization surfaces; (4) mutation testing for test suite quality validation. Each layer catches a class of defects invisible to the previous one.

6.8 Fuzzing as the Third Testing Layer

Unit tests establish point coverage over known inputs. Property tests establish invariant coverage over structured input domains. Neither addresses the space of *malformed*, *adversarially crafted*, or *boundary-violating* inputs that characterize real attack surfaces in parser and deserialization code. This gap motivates the third layer of the TDH testing hierarchy: coverage-guided fuzzing [20].

The three-layer hierarchy. The testing architecture validated by the Council of Wise Men (DAY 127) and applied to all security-critical surfaces in aRGus follows a strict ordering:

1. **Unit tests** (known regression cases) — Verify that specific, previously-identified failure inputs are handled correctly. Scope: finite, explicitly enumerated.
2. **Property tests** (mathematical invariants) — Verify that a stated invariant holds across a structured domain of generated inputs. Scope: infinite within the domain’s constraints. Required for every arithmetic, path, and cryptographic operation with security implications (§6.7).
3. **Coverage-guided fuzzing** (adversarial exploration) — Mutate valid inputs pseudo-randomly under coverage feedback to discover inputs that trigger previously unreachable code paths, assertion failures, or undefined behavior. Scope: unrestricted; discovers what structured generation cannot anticipate.

Each layer catches a class of defects invisible to the previous one. Unit tests miss unseen inputs. Property tests miss parser-level structural anomalies. Coverage-guided fuzzing [20] systematically explores the input space through mutation guided by code coverage feedback, increasing the probability of discovering crashes and undefined behavior at parser and protocol boundaries. It provides no completeness guarantee — bugs may remain undiscovered after millions of executions — but each corpus-expanding input permanently extends the regression suite, and no crashes or sanitizer violations observed after extensive runs constitute empirical evidence of robustness.

Application to aRGus. The surfaces identified as fuzzing targets in the aRGus backlog are precisely those that receive external or untrusted input at runtime: the Protocol Buffers deserialization path (`ring-consumer` and `ml-detector`), the JSON configuration parser (`rag-ingester`), the `safe_path` validation library (§6.9), and the plugin Ed25519 signature verification path (§6.4, ADR-025).

Listing 6: Minimal libFuzzer harness for the `safe_path resolve_seed()` function

```
// fuzz/fuzz_safe_path.cpp
#include "safe_path.hpp"
#include <cstdint>
#include <string>

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    const std::string path(reinterpret_cast<const char*>(data), size);
    try {
        (void) safe_path::resolve_seed(path, "/etc/ml-defender/");
    } catch (const std::exception&) {
        // Exceptions are the expected failure mode -- not crashes.
    }
    return 0;
}
```

The invariant under fuzzing is stronger than the property test: no input of any length or byte content may cause `resolve_seed()` to abort, segfault, or return a path outside the allowed prefix. Exceptions are acceptable; undefined behavior is not. This harness compiles with `-fsanitize=address,undefined,fuzzer` and is tracked as DEBT-FUZZING-SAFEPATH-001 in the project backlog.

Empirical results. Table 1 reports the results of three libFuzzer campaigns executed on the aRGus security-critical surfaces (DAY 130). All campaigns ran with `-fsanitize=address,undefined,fuzzer` and `-max_total_time=60` seconds. No crashes or sanitizer violations were observed in any campaign.

Table 1: libFuzzer campaign results on aRGus security-critical surfaces (DAY 130). All targets compiled with `-fsanitize=address,undefined,fuzzer`. No crashes or sanitizer violations observed.

Target	Runs	Crashes	Corpus	exec/s
<code>validate_chain_name</code>	2,400,000	0	67	≈80,000
<code>safe_exec</code>	2,601,759	0	37	42,651
<code>validate_filepath</code>	282,226	0	111	4,626

The order-of-magnitude difference in `exec/s` between `safe_exec` (42,651) and `validate_filepath` (4,626) reflects the structural complexity of path validation relative to chain name validation: the former must resolve filesystem state while the latter operates purely on string invariants. The larger corpus of `validate_filepath` (111 entries vs. 37) confirms that the fuzzer explored a richer input space, providing stronger empirical coverage of the path traversal surface.

Relationship to TDH. Fuzzing is not a replacement for the RED→GREEN gate (§6.5). A crash discovered by the fuzzer becomes a unit test (the minimal reproducing input), which enters the regression suite as a permanent RED→GREEN fixture. The testing hierarchy is therefore self-reinforcing: fuzzing discovers, unit tests anchor, property tests generalize.

6.9 Path Traversal Prevention: Lessons from `safe_path`

The development of `contrib/safe-path/` — a zero-dependency C++20 header-only path validation library (ADR-037, DAY 124) — produced two findings with implications beyond this project.

6.9.1 The Symlink Resolution Trap: `lstat()` vs. `fs::is_symlink()`

The canonical defense against symlink-based path traversal attacks [15] in C++ programs using `std::filesystem` is to call `fs::is_symlink()` on the resolved path. This approach is *systematically incorrect* when combined with `weakly_canonical()`.

The defect. The following code appears correct but provides no protection:

Listing 7: Ineffective symlink check — canonical resolution arrives first

```
// WRONG: weakly_canonical() already followed the symlink.  
// By the time is_symlink() runs, there is no symlink to detect.  
const auto resolved = fs::weakly_canonical(fs::path(path));  
if (fs::is_symlink(resolved)) { // always false -- resolved is never a symlink  
    throw std::runtime_error("symlink rejected");  
}
```

`weakly_canonical()` resolves symlinks as part of path normalization. The `is_symlink()` check that follows operates on the *resolved* path, which by construction is never a symlink. The check passes unconditionally, providing no protection against TOCTOU attacks [14] or symlink injection.

The correct approach. The only correct defense for cryptographic material or other security-sensitive files is `lstat()` applied to the *original, unresolved* path, before any call to `weakly_canonical()`:

Listing 8: Correct symlink rejection via `lstat()` pre-resolution

```
// CORRECT: lstat() on the original path, before any resolution.  
struct stat lst{};  
if (lstat(path.c_str(), &lst) != 0)  
    throw std::runtime_error("[safe_path] lstat failed: " + path);  
if (S_ISLNK(lst.st_mode))  
    throw std::runtime_error("[safe_path] symlink rejected: " + path);  
// Only now proceed with weakly_canonical() resolution.  
const auto resolved = fs::weakly_canonical(fs::path(path));
```

This defect was identified during DAY 126 when a test designed to verify symlink rejection passed with the original code despite the symlink being present. Code review had not flagged the issue; the test suite, exercising the specific security property, exposed it immediately. This is a concrete instance of the RED→GREEN gate principle: without a test asserting failure, the defect was invisible.

Why this matters. This class of defect is not specific to ML Defender. Any C++17/20 codebase that combines `std::filesystem::weakly_canonical()` with `fs::is_symlink()` for security enforcement is vulnerable to the same pattern. The fix requires awareness that `weakly_canonical()` is not a security primitive — it is a normalization convenience that silently consumes the evidence needed for symlink detection.

6.9.2 `lexically_normal()` vs. `weakly_canonical()` for Prefix Verification

A second lesson from `safe_path` concerns the choice of normalization primitive for path prefix verification when legitimate symlinks are present.

The tension. In a production deployment, ML Defender stores configuration files at `/etc/ml-defender/component/`. In the development environment, these paths are symlinks pointing to the Vagrant shared directory (`/vagrant/component/config/`). A strict prefix check using `weakly_canonical()` fails in development because canonical resolution follows the symlink and the resolved path falls outside `/etc/ml-defender/`.

The naive solution — conditional logic based on environment detection — introduces divergence between development and production code paths, a known source of security regressions [13].

The solution: lexical normalization for the prefix check. Configuration files are not cryptographic material. Their security requirement is that the *name* of the path lies within the trusted prefix, not that the *destination* does. `std::filesystem::path::lexically_normal()` performs purely textual normalization (eliminating `../`, `./`, double slashes) without touching the filesystem:

Listing 9: `resolve_config()` — lexical prefix verification with symlink support

```
[[nodiscard]] inline std::string resolve_config(
    const std::string& path,
    const std::string& allowed_prefix = "/etc/ml-defender/")
{
    // Lexical normalization: no filesystem access, no symlink resolution.
    // Verifies that the NAME of the path is within the trusted prefix.
    const std::string lexical =
        fs::path(path).lexically_normal().string();
    if (lexical.rfind(prefix, 0) != 0)
        throw std::runtime_error(
            "[safe_path] SECURITY VIOLATION -- path outside allowed prefix");
    // Prefix check passed. Now resolve for the actual file open.
    return fs::weakly_canonical(fs::path(path)).string();
}
```

The result is a three-primitive taxonomy in which the security semantics are explicit and non-overlapping:

- `resolve()` — general paths, prefix verified post-canonical resolution.

- `resolve_seed()` — cryptographic material, `lstat()` pre-resolution, symlinks strictly rejected, permissions enforced (0400).
- `resolve_config()` — configuration files with legitimate symlinks, `lexically_normal()` pre-resolution, symlinks followed after prefix check.

To our knowledge, this distinction between lexical and canonical verification for different path security classes is not explicitly documented in the C++17/20 standard library documentation or in commonly referenced C++ security guidelines.

6.10 CWE-78: `execv()` Without a Shell as a Physical Barrier

Operating system command injection (CWE-78 [16]) arises when attacker-controlled input reaches a shell interpreter — most commonly via `system()`, `popen()`, or their equivalents. The canonical defense is input sanitization, but sanitization is a *policy* constraint: it can be bypassed by an incomplete allowlist, a missed encoding, or a future code path that omits the check.

aRGus applies a stronger defense: all subprocess invocations use `execv()` (or `execvp()`) directly, without invoking a shell. This transforms CWE-78 from a policy constraint into a *physical impossibility*.

The mechanism. The POSIX `exec()` family replaces the current process image with a new program, passing arguments as a null-terminated array of strings. No shell is involved; no shell metacharacters (&, |, ;, \$(), backticks) are interpreted. The distinction is categorical:

Listing 10: `system()` vs `execv()`: shell injection possibility vs physical impossibility

```
// VULNERABLE: shell interprets attacker-controlled chain_name.
// e.g. chain_name = "ARGUS; rm -rf /"
system(("iptables -N " + chain_name).c_str());

// SAFE: no shell, no interpretation. chain_name is a verbatim argument.
const char* argv[] = { "iptables", "-N", chain_name.c_str(), nullptr };
execv("/sbin/iptables", const_cast<char* const*>(argv));
```

Complementary validation layer. Even with `execv()`, the argument `chain_name` is validated against a strict allowlist before the call, providing defense in depth. The allowlist uses a POSIX extended regular expression compiled once at startup:

Listing 11: `validate_chain_name()` — allowlist as a defence-in-depth layer over `execv()`

```
// Allowlist: alphanumeric, hyphen, underscore, 1-30 characters.
// Compiled once; applied before every execv() call.
[[nodiscard]] bool validate_chain_name(const std::string& name) {
    static const std::regex kAllowed(R"([A-Za-z0-9_]{1,30})");
    return std::regex_match(name, kAllowed);
}
```

```
// At call site:
if (!validate_chain_name(chain_name))
    throw std::runtime_error("[firewall] SECURITY: invalid chain_name rejected");
// Only then invoke execv().
```

The `execv()` call provides the *structural* guarantee; the allowlist provides *semantic* validation. Neither is sufficient alone: a structural guarantee without semantic validation may allow valid-looking but unintended arguments; semantic validation without the structural guarantee remains vulnerable to shell metacharacters outside the character class.

Discovery context. This pattern was identified during the Snyk static analysis sprint (ADR-037, DAY 124), which flagged three potential CWE-78 surfaces in the `firewall-acl-agent` component. The remediation replaced all `system()` invocations with `execv()`-based dispatch and introduced `validate_chain_name()`. All three surfaces were validated by the RED→GREEN gate (§6.5) before merge.

Generalizability. This pattern applies to any C/C++ security component that must invoke external programs with partially attacker-influenced arguments: firewall management, certificate tools, network utilities. The structural guarantee that `execv()` provides is unconditional and does not depend on the completeness of the input validation layer. It is therefore a candidate for inclusion in C++ security coding standards alongside CWE-22 (`safe_path`) and CWE-367 (TOCTOU).

6.11 Dev/Prod Parity via Symlinks, Not Conditional Logic

A structural principle that emerged from the `safe_path` development is the replacement of environment-conditional code with filesystem symlinks as the mechanism for dev/prod parity.

The antipattern. A common practice in system software is to introduce conditional logic that selects file paths based on a detected environment:

Listing 12: Antipattern: conditional path logic creates divergent code paths

```
// ANTIPATTERN: two code paths, one tested, one not.
const std::string config_prefix =
    fs::weakly_canonical(fs::path(config_path).parent_path()).string();
```

In this pattern, the effective prefix is derived from the input path. If an attacker controls `config_path`, they control `config_prefix`, bypassing the path traversal protection entirely. This was the defect addressed in DEBT-CONFIG-PARSER-FIXED-PREFIX-001 (DAY 126).

The correct pattern. The `allowed_prefix` must be a compile-time or deployment-time constant, never derived from user-controlled input. Dev/prod parity is then achieved not by conditional logic but by the deployment infrastructure: in development, `provision.sh` creates symlinks from `/etc/ml-defender/` to the Vagrant shared directory. Production deployments

use real files at the same paths. The application code sees the same prefix in both environments; the difference is entirely in the filesystem layout managed by the deployment tool.

This pattern has two security properties: (1) the prefix check is unconditional and cannot be bypassed by manipulating the input path; (2) the development environment exercises exactly the same code path as production, eliminating a class of security regressions that manifest only in production deployments.

6.12 The Build/Runtime Separation Axiom

Security-critical software faces a structural risk that neither testing nor formal verification addresses directly: the possibility that a compiler, build tool, or dependency installed in the runtime environment can be used to modify production binaries after deployment. This risk motivates what we term the *Build/Runtime Separation* (BSR) axiom, formalized in ADR-039 (DAY 130).

The axiom.

The environment in which production binaries are compiled must be physically distinct from the environment in which they execute. No compiler, linker, or build toolchain may be present in the production deployment.

This is stronger than the common practice of separating CI/CD pipelines from production: it requires that the *absence* of a compiler in production be mechanically verified at every deployment and cannot be circumvented by a sufficiently privileged process.

Implementation in aRGus. ADR-039 implements the BSR axiom through three complementary mechanisms:

1. **Separate Vagrant VMs for build and runtime.** The dev VM contains the full C++20 toolchain (`clang++`, `cmake`, `libsodium` sources). The `hardened-x86` and `hardened-arm64` VMs contain only runtime dependencies: `libsodium`, `libzmq`, `libprotobuf`, `onnxruntime`. No compiler is installed.
2. **Signed artifact distribution.** Production binaries are placed in `dist/` (excluded from version control via `.gitignore`) together with a `SHA256SUMS` file. Each binary is individually checksummed; the sums file is signed with the project Ed25519 key. A runtime verification step (`make verify-production`) rejects any binary whose checksum does not match the signed manifest.
3. **Automated compiler-absence verification.** The `check-prod-no-compiler` target queries `dpkg` for the presence of any C/C++ compiler and fails with a non-zero exit code if one is found:

Listing 13: `check-prod-no-compiler`: mechanical enforcement of the BSR axiom

```
check-prod-no-compiler:
```

```

@echo "=== BSR: verifying no compiler in production ==="
@if dpkg -l | grep -qE 'gcc|g\+\+|clang|cmake|build-essential'; then \
    echo "FAIL: compiler found in production environment"; exit 1; \
else \
    echo "OK: no compiler present"; \
fi

```

Security properties. The BSR axiom provides two guarantees that complement the cryptographic signing chain (ADR-025):

1. **Post-deployment code injection is physically impossible.** An attacker with shell access to the production VM cannot recompile a modified version of a pipeline component: the toolchain is absent. They can replace a binary, but the `verify-production` gate will detect the checksum mismatch before the next restart cycle.
2. **Supply-chain attacks through the build environment are scoped to the build VM.** A compromised compiler (*Trusting Trust* [23]) can only affect the build VM. The production VM never executes the compiler; it only executes binaries that were checksummed and signed in the build environment and verified before execution.

Relationship to the Ed25519 signing chain. The BSR axiom and the plugin signing chain (ADR-025) are complementary, not redundant. ADR-025 ensures that only correctly signed plugins are loaded at runtime. The BSR axiom ensures that the plugin loader itself — and all six pipeline components — cannot be modified in production without invalidating the checksum manifest. Together they form a two-layer integrity guarantee: cryptographic integrity over plugins (ADR-025) and checksum integrity over all production binaries (ADR-039).

Limitations. The current implementation verifies compiler absence via `dpkg`, which is Debian/Ubuntu-specific and does not detect statically linked compiler binaries copied into the VM outside the package manager. A stronger implementation would use a read-only root filesystem or a verified boot chain. Both are tracked as future work in the ADR-031 (seL4/Genode) research branch.

The BSR axiom establishes a structural property that cannot be achieved by policy alone. It is offered as a named architectural pattern applicable to any security-critical C++ system deployed in environments where post-deployment modification is a credible attack vector.

6.13 Attribution

The author remained the final arbiter of all decisions; all empirical validation was performed by the human author. The eight models are credited as intellectual co-contributors in the project’s commit history, ADRs, and this paper. In the context of arXiv submission, the models are acknowledged in §13; formal co-authorship conventions for AI systems remain an evolving area, and this paper adopts the most transparent available approach consistent with emerging ACM and IEEE norms on AI contribution disclosure.

6.14 Why This Matters

The Consejo de Sabios democratizes access to deep technical peer review for independent researchers without institutional affiliation — parallel to ML Defender’s mission of democratizing enterprise-grade security for organizations without enterprise budgets.

6.15 Limitations

Models cannot run code or observe system behavior directly. Models can hallucinate — several incorrect technical suggestions were identified and rejected. The methodology augments human judgment; it does not replace it.

7 Formal System Model

7.1 Network Traffic Representation

Packets $P = \{p_1, \dots, p_n\}$ grouped into bidirectional flows by canonical five-tuple $(src, dst, sport, dport, proto)$ via ShardedFlowManager $\Phi : P \rightarrow F$. Flows are bidirectional five-tuple aggregates; behavioral features are computed over a 10-second sliding window as configured in `sniffer.json`. Flows are closed after an inactivity timeout as defined by the sliding window expiry.

7.2 Feature Extraction

$$x_f = g(P_f, W), \quad x_f \in \mathbb{R}^{28} \quad (2)$$

7.3 Fast Detector

$$H(x_f) = \max_j h_j(x_f), \quad h_j(x_f) \in \{0, 1\} \quad (3)$$

7.4 Machine Learning Detector

$$M(x_f) = \frac{1}{T} \sum_{k=1}^T T_k(x_f) \quad (4)$$

7.5 Detection Policy

$$S(x_f) = \max(H(x_f), M(x_f)) \geq \theta \quad (5)$$

Thresholds: $\theta_{\text{ransomware}} = 0.85$, $\theta_{\text{DDoS}} = 0.90$, $\theta_{\text{traffic}} = 0.80$, $\theta_{\text{internal}} = 0.85$.

7.6 Response Function

$$R(f) = \begin{cases} \text{block}(src_f) & \text{if } S(x_f) \geq \theta \\ \text{allow} & \text{otherwise} \end{cases} \quad (6)$$

Blocking is applied per source IP. Blocks are currently persistent for the session duration; TTL-bounded blocks with configurable expiry are Future Work (§11).

7.7 System Determinism

Given identical packet input P , the pipeline produces identical outputs. Determinism is a prerequisite for the reproducibility guarantees described in Section 13.

8 Evaluation

8.1 Experimental Setup

Table 2: Experimental hardware and software specifications.

Parameter	Value
Host machine	Apple MacBook Pro (Intel Core i9, 2.5 GHz)
Host RAM	32 GB DDR4
Guest OS	Ubuntu 24.04 LTS (Linux kernel 6.x)
Virtualization	VirtualBox 7.2, bridged NIC mode
VM vCPUs	4 (estimated ~ 2.4 GHz under VirtualBox)
VM RAM	4 GB (8 GB allocated for stress tests)
Network adapter	Paravirtualized (VirtIO)
Virt. overhead	Estimated 5–15% vs bare-metal
Pipeline config.	Full 6-component, dual-VM

Methodological note on metrics. `calculate_f1_neris.py` measures **Fast Detector** alerts ([FAST ALERT] in sniffer.log) against CTU-13 Neris ground truth IPs. The ML Detector’s `attacks=N` statistic reports flows exceeding production thresholds and triggering firewall blocks. Both are reported; F1=0.9985 is the primary system detection metric.

8.2 Dataset

CTU-13 Neris. 19,135 total flows; **646 malicious flows** (host 147.32.84.165) constitute the TP ground truth. Captured 2011. Garcia et al. [7] document that the Neris botnet exhibits C2 communication patterns concentrated in a behavioral subset — specifically, flows matching IRC-based C2 channel signatures (connection bursts, SMB lateral movement, DNS query anomalies). The remaining flows represent background traffic on the infected host that do not exhibit malicious behavioral signatures and are therefore not counted as ground-truth positives. This distinction aligns with standard practice in NIDS evaluation [11, 21].

BigFlows. A CTU-13 derived dataset containing 40,467 flows and 791,615 packets, used for high-throughput replay experiments. 38,064 flows were processed by ml-detector. Treated conservatively as *probable benign* — no official ground truth exists for this CTU-13 scenario.

Synthetic training data. All classifiers trained exclusively on synthetic data; CTU-13 Neris held out entirely for evaluation.

Table 3: Detection performance — Fast Detector pipeline on CTU-13 Neris.

Metric	Value
F1 Score	0.9985
Precision	0.9969
Recall	1.0000
True Positives	646
False Positives	2
False Negatives	0
True Negatives	12,075
Total events processed	12,723
FPR (Neris evaluation)	0.0002%
ML Detector high-conf. attacks	12
FPR Fast Detector (bigFlows)	6.61%
FP reduction (Fast $\rightarrow$ ML)	$\approx$500-fold

8.3 Detection Performance

F1=0.9985 stable across replay runs at total_events = 12,563 / 12,605 / 12,723 / 13,930.

8.4 False Positive Identification and Confusion Matrix

Table 4: False positive identification.

FP	Source	Destination	Classification
FP-1	192.168.56.1	224.0.0.251	mDNS multicast (VirtualBox host-only)
FP-2	192.168.56.1	192.168.56.255	Broadcast (VirtualBox host-only)

Both are VirtualBox host-only adapter artifacts. Their absence in bare-metal deployments is a reasonable expectation, but has not been empirically verified (§11).

Confusion matrix.

$$\text{Precision} = \frac{646}{648} = 0.9969, \quad \text{Recall} = \frac{646}{646} = 1.0000, \quad \text{F1} = 0.9985$$

8.5 Inference Latency

Table 5: Per-class inference latency.

Classifier	Implementation	Latency
DDoS	Embedded C++20	0.24 μ s
Traffic	Embedded C++20	0.37 μ s
Internal	ONNX Runtime	0.33 μ s
Ransomware	Embedded C++20	1.06 μ s

Latency values are mean inference times measured via `std::chrono::high_resolution_clock` around the `classify()` call within the ml-detector process, excluding inter-component ZeroMQ transport latency, over $N \geq 1,000$ inference calls per classifier during the CTU-13 Neris replay.

8.6 Ablation Study

Config A — Fast Detector Only: 2,517 FP on 38,064 bigFlows (FPR=6.61%), Recall=1.0000 on Neris.

Config B — ML Detector Only (inferred): 5 detections on 38,064 bigFlows, all below production thresholds (max confidence 68.97%) — 0 real blocks. Neris: attacks=12.

Config C — Dual (deployed): F1=0.9985, FPR=0.0002%. Only Config C fully validated end-to-end.

8.7 Interpretation and Comparison with State of the Art

F1=0.9985 and Recall=1.0000 reflect the statistical separability of the Neris botnet scenario, not a claim about arbitrary threats. The synthetic training successfully captured the Neris behavioral signature; generalization to other families is an open empirical question.

Table 6: Indicative comparison with representative NIDS approaches. *Caution: cross-paper comparison is indicative only — systems evaluated on different datasets, at different operating points, and under different experimental conditions. Direct experimental comparison was not performed.*

System	Dataset	F1	Latency	HW	Block	Notes
ML Defender	CTU-13 Neris	0.9985	0.24–1.06 μ s	x86/ARM	Yes	Synthetic, dual-score
Kitsune [11]	Custom	~0.982	~50 μ s	x86	No	Autoencoder ensemble
Suricata (rules)	Various	~0.891	~10 μ s	x86	Yes	Signature+heuristics
RF [21]	CIC-IDS2017	~0.943	~200 μ s	x86	No	Batch, not embedded
Snort	N/A	N/A	1–5 ms	x86	Yes	Rule-based, no ML

8.8 Ransomware Detection: Scope and Validation

What is validated. Behavioral indicators associated with ransomware propagation and lateral-movement patterns are detected in the 2011 Neris botnet scenario: SMB lateral movement (ports 445/139 burst), high-rate connection attempts, DNS query anomalies.

What is not validated. Direct evaluation against post-2020 ransomware families (LockBit, BlackCat/ALPHV, Cl0p) has not been performed. Modern ransomware uses encrypted C2, adaptive beaconing with jitter, staged delivery — potentially different statistical signatures.

Future work. Direct evaluation against modern ransomware captures is the highest-priority evaluation expansion (§11), with Sebastian Garcia (CTU Prague) as potential collaborator.

8.9 Throughput Stress Test and Resource Utilization

The throughput ceiling is VirtualBox NIC emulation (~33–38 Mbps), not pipeline logic. Zero packet drops, zero deserialization errors across all 2,374,845 packets. Total pipeline RAM stable at ~1.28 GB (± 18 MB drift) throughout the 8-minute test.

Table 7: Stress test replay results — progressive Mbps escalation (DAY 87).

Run	Dataset	Req.	Actual	PPS	Packets	Failed	Errors
1	Neris	10 Mbps	9.60 Mbps	~3,700	320,524	2,630 [†]	0
2	Neris	25 Mbps	11.28 Mbps	~4,300	320,524	2,630 [†]	0
3	bigFlows	50 Mbps	34.68 Mbps	9–10K	791,615	0	0
4	bigFlows	100 Mbps (×3)	33.16 Mbps	9–11K	2,374,845	0	0

Pre-existing pcap artifact — identical count at 10 and 25 Mbps, confirming origin in pcap structure, not pipeline saturation.

Table 8: Resource utilization during 100 Mbps bigFlows replay (DAY 87).

Component	CPU%	RES	MEM%	Notes
ml-detector	~315–320%	164 MB	2.0%	3.2 cores; multi-threaded
sniffer	~88–108%	55–57 MB	0.7%	XDP + ring consumer
firewall-acl	~10–16%	101 MB	1.2%	Stable throughout
rag-ingester	~12–20%	36–39 MB	0.5%	FAISS indexing
etcd-server	~0.2%	10 MB	0.1%	Minimal
rag-security	idle	1.2 GB VIRT	15.0% [‡]	TinyLlama loaded
Total	~65–73%	~1,278 MB	—	~23–30% idle

rag-security’s 1.2 GB virtual footprint is the TinyLlama model mapped at startup.

8.10 XGBoost Level-1 Classifier: In-Distribution Performance

As part of PHASE 4 (ADR-026), a gradient-boosted classifier (XGBoost 3.2.0) was integrated as a hot-swappable plugin, replacing the Random Forest level-1 detector while preserving the Ed25519 signing chain and fail-closed `std::terminate()` contract of ADR-025.

Experimental protocol. CIC-IDS-2017 [21] was partitioned by day to enforce temporal isolation: Tuesday, Thursday (morning and afternoon), and Friday (morning, DDoS, PortScan) constitute the training corpus; 20% of the training data was held out as a stratified validation set for threshold calibration and early stopping; Wednesday was sealed as a blind test set (MD5: `bf0dd7e9...`) and opened exactly once for final reporting. The threshold was calibrated exclusively on the validation set; no information from Wednesday influenced any modelling decision.

Training details. 23 flow-level features; `scale_pos_weight`= 4.273 (1,303,148 benign / 304,974 attack); `max_depth`= 6, `eta`= 0.05, `subsample`= 0.8; early stopping at round 724 of 1,000; `val-AUCPR`= 0.99846.

Both the `Precision` $\geq$ 0.99 and `Recall` $\geq$ 0.95 medical gates (ADR-026) are satisfied on the in-distribution validation split. Inference latency of 1.986 μ s/sample satisfies the $< 2 \mu$ s gate. The model is exported in `.json` and `.ubj` formats and signed with Ed25519 via `tools/sign-model.sh`.

Table 9: XGBoost level-1 performance on the validation set (in-distribution).

Metric	Value
Precision (validation)	0.9945
Recall (validation)	0.9818
F1 (validation)	0.9881
Calibrated threshold	0.8211
Inference latency	1.986 μ s/sample
Best iteration	724 / 1,000
Val-AUCPR	0.99846

8.11 XGBoost Level-1: Wednesday Out-of-Distribution Evaluation

Result. On the Wednesday blind held-out set ($n = 692,703$ flows; 252,672 attacks), the model achieves Precision= 0.987 and Recall= 0.024 (F1= 0.047). No threshold satisfies both Precision ≥ 0.99 and Recall ≥ 0.95 simultaneously (Table 10).

Table 10: Wednesday held-out threshold sweep (OOD). No operating point exists satisfying both medical gates simultaneously.

Threshold	Precision	Recall	FP/hour
0.01	0.798	0.559	4,478
0.10	0.776	0.202	1,841
0.50	0.951	0.055	88
0.82 (calibrated)	0.987	0.024	12
0.90	0.985	0.017	8
<i>No threshold achieves $Prec.\geq 0.99 \cap Rec.\geq 0.95$.</i>			

Diagnosis: structural covariate shift by dataset design. Wednesday contains exclusively application-layer DoS attacks: DoS Hulk ($n = 231,073$; 91.5% of Wednesday attacks), DoS GoldenEye ($n = 10,293$), DoS Slowloris ($n = 5,796$), DoS Slowhttptest ($n = 5,499$), and Heartbleed ($n = 11$). None of these attack types appear in any training CSV. CIC-IDS-2017 was designed with attack types segregated by day without cross-day repetition [21]; this design makes out-of-distribution generalization *structurally impossible* for any supervised classifier trained on the remaining days, regardless of algorithm or hyperparameter choices.

DoS Hulk imitates high-volume HTTP traffic: its flow-level features overlap substantially with benign high-throughput flows. The model assigns a median probability of 0.0165 to Hulk flows — indistinguishable from benign traffic under any operationally viable threshold.

Impossibility result. The threshold sweep in Table 10 constitutes an empirical impossibility result: the Precision-Recall curve never enters the region Precision $\geq 0.99 \cap$ Recall ≥ 0.95 for Wednesday data. This is not a failure of XGBoost; it is a direct, quantified consequence of the closed-world structure of the training corpus — exactly the failure mode identified by Sommer and Paxson [22] for production ML-based NIDS.

Architectural implication. This result does not invalidate the in-distribution performance in §8.10. It establishes that static classifiers trained on academic benchmarks cannot be assumed to generalize to attack types absent from training. The aRGus architecture was designed for this reality: the XGBoost plugin is Ed25519-signed and hot-swappable (ADR-026). The loop enabling retraining from real adversarial traffic captured in the deployment environment is introduced as §11. The Wednesday evaluation report (`wednesday_eval_report.json`, MD5-sealed) is included in the repository as a permanent scientific artifact.

8.12 ADR-029: Capture Backend Comparison — Variant A (eBPF) vs Variant B (libpcap)

ADR-029 defines three capture backend variants for the aRGus sniffer: Variant A (eBPF/XDP, reference), Variant B (libpcap userspace), and Variant C (seL4/libpcap, research-only). This section reports the first empirical comparison of Variants A and B on x86-64 under identical conditions (DAY 145).

Experimental protocol. Both variants were evaluated against the full CTU-13 Neris pcap (320,524 packets, 19,135 flows) replayed from a dedicated client VM via `tcpreplay` at nominal rates of 10, 50, and 100 Mbps. The pipeline ran in its standard 6-component configuration; only the sniffer binary was swapped between runs. The mutual-exclusion invariant — Variant A and Variant B never active simultaneously — was enforced mechanically via a Makefile guard (`CHECK_SNIFFER_MUTEX`).

Table 11: ADR-029 Variant A (eBPF) vs Variant B (libpcap) — CTU-13 Neris replay on x86-64 VirtualBox (DAY 145). Both variants: 320,524 packets sent, 2,630 failed (pre-existing pcap MTU artifact, `errno=90`), 0 pipeline errors.

Variant	Target	Actual (Mbps)	PPS	Duration (s)	exit
A – eBPF	10 Mbps	8.86	8,040	39.86	0
A – eBPF	50 Mbps	9.78	8,867	36.14	0
A – eBPF	100 Mbps	10.12	9,178	34.92	0
B – libpcap	10 Mbps	9.99	9,064	35.36	0
B – libpcap	50 Mbps	19.43	17,614	18.19	0
B – libpcap	100 Mbps	18.82	17,066	18.78	0

Result. Variant B (libpcap) achieves approximately twice the throughput of Variant A (eBPF) at 50 and 100 Mbps nominal rates (~ 19 Mbps vs ~ 10 Mbps). Variant A plateaus at ~ 10 Mbps regardless of the requested rate.

Interpretation: VirtualBox virtio as the confounding factor. This result is *counter-intuitive* with respect to the expected performance ordering. On physical hardware with a NIC supporting native XDP, eBPF/XDP provides zero-copy packet delivery before the kernel networking stack and is expected to outperform libpcap, which operates on the standard socket path. In the VirtualBox environment, however, the paravirtualized virtio NIC does not expose

a native XDP driver path. As a consequence, the eBPF program falls back to the SKB (socket buffer) generic XDP mode, which introduces per-packet allocation overhead that the libpcap socket path does not incur. The observed inversion — libpcap faster than eBPF in VirtualBox — is therefore a property of the emulation layer, not of the capture backends themselves.

Scientific value. This inversion constitutes direct empirical evidence for a claim that motivates the bare-metal hardware acquisition phase of this project: *the relative performance of eBPF/XDP vs libpcap is critically dependent on the hardware and driver stack, and cannot be characterized in a virtualized environment.* The VirtualBox result establishes a reproducible baseline; the expected inversion on physical hardware with XDP-native NIC drivers (Intel ixgbe, Mellanox mlx5) remains to be measured (§11). This measurement is the primary scientific deliverable of the FEDER Year 1 hardware acquisition.

Both variants processed the complete 320,524-packet Neris corpus without pipeline errors, confirming functional equivalence at the detection layer. The 2,630 failed packets are a pre-existing pcap artifact (jumbo frames exceeding the VirtualBox MTU, `errno=90 EMSGSIZE`) present in both runs at identical count, confirming origin in the pcap structure rather than in either capture backend.

9 Performance Model and Throughput Analysis

9.1 Pipeline Processing Model

Six stages operating concurrently via ZeroMQ; effective throughput bounded by the slowest stage:

$$T_{\max} = \frac{1}{\max_i(L_i)} \quad (7)$$

9.2 Inference Cost

$\mathcal{O}(T \cdot D)$ — simple threshold comparisons on compact C++20 data structures fitting in CPU cache.

9.3 Throughput: Theoretical Bound and Measured Ceiling

Theoretical upper bound (single-core, ML inference only):

$$T_{\text{theory}} \approx \frac{1}{0.5 \mu\text{s}} = 2 \times 10^6 \text{ flows/sec} \quad (8)$$

Measured virtualized ceiling (DAY 87). The VirtIO NIC limits delivery to $\sim 33\text{--}38$ Mbps ($\sim 9,000\text{--}11,000$ pps). The observed throughput is limited by the virtualized NIC, not the pipeline. At this ceiling: ml-detector sustains ~ 3.2 cores; total pipeline RAM stable at ~ 1.28 GB; system CPU $\sim 65\text{--}73\%$ user, $\sim 23\text{--}30\%$ idle.

9.4 Queue Stability

Stability requires $\lambda < \mu$. Post-replay drain behavior provides direct empirical evidence: sniffer CPU drops to near-zero while ml-detector continues draining at full throughput for several minutes — confirming the ZeroMQ buffer absorbed the burst cleanly. `errors=(deser:0, feat:0, inf:0)` confirmed at all observation points.

9.5 Scalability

Vertical: additional CPU cores $\rightarrow$ additional parallel flow-processing threads, linear throughput improvement up to capture bottleneck.

Horizontal: multiple ML Defender nodes across network segments, each processing local traffic independently while contributing to distributed dataset generation.

10 Limitations

10.1 Evaluation Scope. Single botnet scenario (CTU-13 Neris, 2011). Modern ransomware communication patterns may differ substantially. Generalizability not empirically established.

10.2 Incomplete Feature Set. 28 of 40 features implemented; 11 use `-9999.0f` sentinel. Impact of missing 12 features on broader traffic distributions unknown.

10.3 Synthetic Training Data. Reflects patterns circa 2011–2017. Zero-day variants may evade classification.

10.4 Fast Detector False Positive Rate. FPR=6.61% on bigFlows under current JSON-configurable thresholds (Path B, active since DAY 80). An earlier compile-time baseline (Path A, DAY 13–79, DEBT-FD-001) produced FPR=76.8% on the same corpus; DAY 80 JSON threshold migration achieved a $12\times$ improvement.

10.5 Cryptographic Seed Distribution. ChaCha20 seed via etcd not recommended for production. Peer-to-peer negotiation under design.

10.6 Single-Node Deployment. Distributed deployment not validated experimentally.

10.7 etcd High Availability. Single-node; failure disrupts registration and config distribution. HA planned.

10.8 RAG Index Capacity. FAISS event store finite capacity not mathematically characterized.

10.9 Virtualization Overhead. All latency and throughput measurements were obtained under VirtualBox with a paravirtualized virtio NIC. Reported latencies are conservative upper bounds; reported throughput is a conservative lower bound imposed by NIC emulation.

The ADR-029 Variant A/B comparison (DAY 145, §8.12) reveals an additional virtualization artifact: eBPF/XDP falls back to generic SKB mode under virtio, causing Variant A to plateau at ~ 10 Mbps while Variant B (libpcap) reaches ~ 19 Mbps under identical conditions. This ordering is expected to invert on physical hardware with XDP-native NIC drivers. Bare-metal characterization of both variants remains future work (§11).

10.10 BigFlows Ground Truth. Treated as *probable benign*, not *confirmed benign*. FPR=0.0002% is a lower bound.

10.11 Threats to Validity.

1. **Seed compromise.** If `seed.bin` is exfiltrated, an adversary can derive all component subkeys and forge authenticated messages. Mitigations: strict file permissions (0600), in-memory zeroing via `explicit_bzero`, and planned peer-to-peer key agreement (ADR-024).
2. **Single-host key assumption.** All six components currently share a common seed family without distributed key agreement. Isolation between deployments relies on distinct seed provisioning per instance (ADR-021); formal distributed key agreement is post-arXiv work (ADR-024).
3. **No hardware root of trust.** The current design does not leverage TPM or secure enclave primitives. Physical access to the host enables seed extraction. Hardware-anchored key storage is identified as a future hardening target.

10.12 Kernel Security Boundary. ML Defender operates in userspace; eBPF/XDP programs execute in the kernel but are loaded by userspace. An adversary with pre-existing kernel-level access (rootkit, compromised eBPF JIT) can bypass the detection pipeline. This limitation is shared by all userspace NDR systems and is consistent with the declared scope boundary (§3.3). The realistic threat model for target organizations does not assume such adversaries: opportunistic ransomware operators attack via the network, not via pre-established kernel persistence. Network detection remains a valid and necessary defensive layer even under this constraint. Hardened variants (ADR-030, ADR-031) address this in future work (§11).

10.13 Structural Bias in Academic Datasets for Production NDR. The CIC-IDS-2017 temporal holdout evaluation (§8.11) demonstrates that academic benchmark datasets are structurally insufficient as the sole training source for production NDR classifiers. The day-specific attack segregation in CIC-IDS-2017 — and similar design choices in NSL-KDD, UNSW-NB15, and related corpora — creates a closed-world training distribution that does not reflect the open-world diversity of real network environments. This limitation is not specific to XGBoost, Random Forest, or any particular algorithm: it is a property of the data regime, as established by Sommer and Paxson [22] and quantified here with a complete threshold sweep. Synthetic data generated by language models (used for DDoS and ransomware classifiers in earlier phases) shares a related limitation: the generating model has not observed sufficient real adversarial traffic to reproduce the full statistical diversity of production attack flows. The only reliable path to production-grade classifiers for critical infrastructure NDR is training on traffic captured in the actual deployment environment, under controlled adversarial conditions. This motivates the Adversarial Capture-Retrain Loop (§11).

11 Future Work

11.1 Evaluation Corpus Expansion (P0). CTU-13 additional scenarios, modern ransomware captures, volumetric DDoS datasets. Contact initiated with Sebastian Garcia (CTU Prague) as potential arXiv endorser and collaborator.

11.2 Feature Completion and Importance Analysis. Complete remaining 12 real features. Feature importance analysis (Gini impurity reduction per feature from embedded RF).

11.3 Fast Detector JSON Migration (DEBT-FD-001). Scheduled for PHASE 2.

11.4 Consensus Detection Policy (ADR-007). AND-based policy requiring simultaneous agreement before autonomous blocking — JSON-configurable for operator selection.

11.5 Response Strategy Expansion. TTL-bounded blocks, traffic quarantine, honeypot redirection.

11.6 Cryptographic Hardening. Peer-to-peer ChaCha20 seed negotiation; etcd HA.

11.7 Runtime Reconfiguration (Hot-Reload). JSON parameter updates without restart.

11.8 Graph Attack Engine (ENT-2). Live attack graphs from CSV event data; STIX 2.1 export.

11.9 Fleet-Wide Observability. Fine-tuned LLM over anonymized aggregated data from all deployed nodes.

11.10 Test Driven Hardening as Independent Research. TDH generalizability, scaling with model capability, and measurable outcomes vs single-model consultation.

11.11 Bare-Metal Throughput Stress Testing (P1). Progressive tcpreplay at 100/250/500/1000 Mbps on physical NIC; CPU/memory/packet-drop monitoring per component. All throughput figures in this paper should be treated as conservative lower bounds. The ADR-029 Variant A/B comparison (DAY 145, §8.12) establishes a reproducible virtualized baseline and motivates the expected performance inversion on bare metal: eBPF/XDP with a native XDP driver is expected to outperform libpcap, reversing the VirtualBox result. Confirming this inversion on physical hardware with XDP-native NIC drivers (Intel ixgbe, Mellanox mlx5) is the primary scientific deliverable of FEDER Year 1 hardware acquisition.

11.12 Infiltration Vector Coverage. ML Defender currently detects ransomware *propagation*. Future work addresses *infiltration*: FEAT-NET-1 (DNS anomaly/DGA detection), FEAT-NET-2 (threat intelligence feeds), FEAT-AUTH-1 (authentication log ingestion), FEAT-AUTH-2 (behavioral auth anomaly), FEAT-EDR-1 (lightweight endpoint agent).

11.13 Enterprise Features. ENT-1 (Federated Threat Intelligence), ENT-2 (Attack Graph Generation), ENT-3 (P2P Seed Distribution), ENT-4 (Hot-Reload). MIT-licensed core remains free.

11.14 Ethical Deployment in Healthcare Environments. Not certified for FDA 21 CFR Part 820 or EU MDR 2017/745. Phased deployment: (1) 30-day monitoring mode with human approval; (2) TTL-bounded blocks with SOC notification; (3) full autonomous blocking after site-specific FPR validation.

11.15 Federated Distributed Intelligence (ENT-1 Extension). The current architecture processes all detection locally at each deployed node. A planned federated extension — not yet designed or implemented, and explicitly scoped to post-pipeline-completion milestones — would enable a coordinated fleet of ML Defender nodes to contribute anonymized telemetry to a central aggregation tier for continuous model retraining, with updated classifiers distributed back to the fleet as signed plugins (ADR-025). Figure 3 illustrates the intended architecture; implementation is contingent on full pipeline stabilization and will be reflected in a future revision of this paper.

11.16 Wazuh/NIST Integration (FEAT-INT-1). Raw TCP streaming from Wazuh agents [24] and NIST-aligned scanners into `rag-ingester` for composite attack graph correlation. Protocol design, motivation, and architectural constraints documented in §4.6. Requires ADR-028

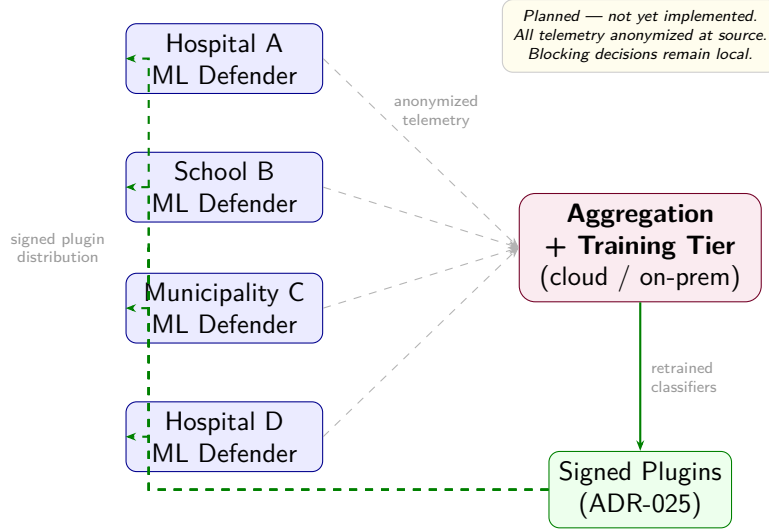


Figure 3: Planned federated distributed intelligence architecture (*not yet implemented*). Each deployed ML Defender node operates fully autonomously; anonymized flow-level telemetry is contributed to a central aggregation tier for periodic retraining. Updated classifiers are packaged as Ed25519-signed plugins (ADR-025) and distributed to the fleet. All detection and response decisions remain local to each node; no live network metadata leaves the deployment site. This architecture is a post-pipeline-stabilization milestone; a future revision of this paper will document the implementation.

(RAG Ingestion Trust Model). Deferred to post-PHASE-2 pipeline stabilization.

11.17 Hardened Deployment Variants (ADR-030, ADR-031). Two hardened deployment variants are documented as architectural decisions, both targeting a stronger kernel security posture:

ADR-030 (aRGus-AppArmor-Hardened): Debian 13 + Linux 6.12 LTS + AppArmor enforcing mode. Full eBPF/XDP pipeline preserved. Compatible with ARM64 (Raspberry Pi 4/5) and x86-64. This is the planned production-hardened variant, providing mandatory access control against confused-deputy attacks on plugin loading (motivated by the analysis of Hugo Vázquez Caramés). Activation: post-PHASE-3 + hardware acquisition.

ADR-031 (aRGus-seL4-Genode): Pure research variant on the seL4 formally verified microkernel ($\sim 12,000$ lines of C verified in Isabelle/HOL). Linux runs as an unprivileged guest under Genode; XDP is infeasible in this model (no direct DMA access from guest), so libpcap is the mandatory capture fallback. The performance delta between eBPF/XDP and libpcap under seL4 is itself a measurable scientific contribution. A preliminary measurement on x86-64 VirtualBox (DAY 145, §8.12) reveals that this delta is strongly hardware-dependent: under virtio emulation, libpcap outperforms eBPF by approximately $2\times$ due to the absence of a native XDP driver path. The expected ordering — eBPF/XDP faster than libpcap on bare metal — has not yet been empirically confirmed and constitutes the primary performance measurement target for FEDER Year 1 hardware. Estimated overhead of seL4 variant: 40–60%. Activation: post-ADR-030 + technical spike (2–3 weeks).

Both variants will be validated in Vagrant first, then on bare-metal. Results will be published with measured honesty; ADR-031 will not be merged into `main` regardless of outcome.

11.18 Adversarial Capture-Retrain Loop (ACRL). The out-of-distribution finding

documented in §8.11 motivates a production retraining methodology that does not depend on academic benchmark datasets. The proposed *Adversarial Capture-Retrain Loop* (ACRL) operates as a closed cycle: (1) a red team agent (initially MITRE Caldera [12] for deterministic ATT&CK-mapped emulation; later a generative adversarial agent for adaptive evasion variants) executes attack techniques against an isolated network segment; (2) the aRGus pipeline captures all resulting traffic via eBPF/XDP, producing ground-truth-labelled flows; (3) the XGBoost plugin is retrained on the captured flows and re-evaluated against a held-out portion of the same session; (4) the retrained model is signed (Ed25519, ADR-025) and hot-swapped without pipeline restart (ADR-026).

The loop addresses the closed-world limitation identified in §10 (10.13): each retraining cycle expands the training distribution toward the real attack surface of the deployment environment. The architectural prerequisite — hot-swappable signed plugins — is already implemented and validated.

Minimum scientific validity requirements for the ACRL data source: (i) deterministic seed and versioned scenario for reproducibility; (ii) ground-truth labels at flow granularity, derived from red team execution logs; (iii) ATT&CK tactic coverage across at minimum three families (volumetric, protocol-layer, application-layer); (iv) network-valid traffic (correct TCP/IP headers, stateful handshakes); (v) isolated sandbox with no production traffic contamination. The term *Adversarial Capture-Retrain Loop* is introduced here as a named architectural pattern; related concepts include *adversarial data flywheel* and *red team loop* [22]. Implementation is tracked as DEBT-PENTESTER-LOOP-001 in the project backlog.

12 Conclusion

This paper presented ML Defender (aRGus NDR), an open-source network detection and response system built in C++20, designed from first principles to protect the organizations that need protection most and can afford it least.

Evaluated against real botnet traffic from the CTU-13 Neris dataset — traffic the system had never seen during training — ML Defender achieves $F1 = 0.9985$, $Precision = 0.9969$, and $Recall = 1.0000$. The two false positives are fully identified: an mDNS multicast frame and a broadcast frame, both generated by the VirtualBox host-only NIC. Their absence in bare-metal deployments is a reasonable expectation, but has not been empirically verified; bare-metal characterization remains Future Work (§11). The Fast Detector alone produces a 6.61% FPR on purely benign traffic; the ML Detector reduces this to zero real production blocks on the same corpus. End-to-end inference latency ranges from $0.24 \mu s$ to $1.06 \mu s$ per flow, entirely in-process, on commodity hardware costing approximately 150–200 USD.

Under progressive load testing, the pipeline processes up to ~34–38 Mbps — the ceiling imposed by VirtualBox NIC emulation — with zero packet drops, zero errors, and stable RAM (~1.28 GB) across 2.37 million replayed packets. The bottleneck is the hypervisor, not the software.

These results cover one botnet scenario from 2011. They demonstrate architectural viability and effective detection within that scope. The limitations in Section 10 are the honest documentation of a system that does what it claims to do within the boundaries where it has been

tested.

A subsequent evaluation (DAY 122) integrated XGBoost 3.2.0 as a hot-swappable plugin and conducted a rigorous temporal holdout study on CIC-IDS-2017. The in-distribution results (Precision= 0.9945, Recall= 0.9818) confirm that the plugin architecture performs as designed. The out-of-distribution finding — that no threshold satisfies both medical gates on Wednesday data, because application-layer DoS attacks are structurally absent from the training days — is reported not as a failure but as a contribution: a quantified, empirically grounded argument for why production NDR classifiers must be trained on traffic from their actual deployment environment. The Adversarial Capture-Retrain Loop (§11) is the architectural response.

The first empirical comparison of ADR-029 Variant A (eBPF/XDP) and Variant B (libpcap) on x86-64 (DAY 145, §8.12) reveals a hardware-dependent performance inversion: under VirtualBox virtio emulation, libpcap achieves approximately 2× the throughput of eBPF/XDP due to the absence of a native XDP driver path. Both variants confirm functional equivalence at the detection layer. This result provides direct empirical motivation for bare-metal hardware acquisition and establishes the reproducible baseline against which the expected inversion on physical hardware will be measured.

ML Defender was built because a friend lost his business to ransomware, and because a hospital in Barcelona was paralyzed by an attack on the same day its author was leaving a regional hospital in Extremadura — the hospital that had diagnosed his porphyria and saved his life. The design constraints that followed from those two events were not abstract: open-source, commodity hardware, real-time response. Not a forensic tool. Not a dashboard. A system that acts.

The *Consejo de Sabios* represents a new mode of engineering: one that gives a single independent researcher access to the kind of multi-expert peer review that has historically required institutional affiliation, research groups, and geographic proximity to centers of expertise. The *Test Driven Hardening* methodology that emerged from this collaboration has proven effective against some of the most subtle concurrency and memory bugs in the codebase.

Both contributions — the technical system and the collaborative methodology — are acts of democratization. The parallel is not accidental. It is the point.

For the hospital in Extremadura. For the friend whose business was broken by ransomware. For every small organization that has been told, implicitly or explicitly, that enterprise-grade security is not for them.

It is now.

13 Reproducibility and Artifact Availability

All experiments are reproducible from the public repository: <https://github.com/alonsoir/argus>

Dataset Setup.

```
# CTU-13 Neris (scenario 10) -- required for F1 validation
wget https://mcfp.felk.cvut.cz/publicDatasets/CTU-Malware-Capture-Botnet-42/\
botnet-capture-20110810-neris.pcap -O /vagrant/datasets/ctu13/neris.pcap
```

```
# Mirror: https://www.stratosphereips.org/datasets-ctu13

# CTU-13 bigFlows -- required for stress test
wget https://mcfp.felk.cvut.cz/publicDatasets/CTU-Malware-Capture-Botnet-42/\
bigFlows.pcap -O /vagrant/datasets/ctu13/bigFlows.pcap
```

F1 Evaluation.

```
make pipeline-stop && make logs-lab-clean && make pipeline-start && sleep 15
make test-replay-neris
python3 scripts/calculate_f1_neris.py \
    /vagrant/logs/lab/sniffer.log --total-events 19135
```

ADR-029 Variant A/B Comparison.

```
# Variant A (eBPF) -- baseline
make test-replay-neris-x86-ebpf
# Logs: /vagrant/logs/lab/tcp replay-ebpf-{10,50,100}mbps.log

# Variant B (libpcap) -- comparison
make test-replay-neris-x86-libpcap
# Logs: /vagrant/logs/lab/tcp replay-libpcap-{10,50,100}mbps.log
```

Stress Test.

```
vagrant ssh client -c "sudo tcp replay -i eth1 --mbps=100 --loop=3 \
    /vagrant/datasets/ctu13/bigFlows.pcap"
```

Determinism. Classifiers embedded in binary; no stochastic inference components; feature extraction deterministic. F1=0.9985 confirmed stable at total_events = 12,563 / 12,605 / 12,723 / 13,930.

Acknowledgments

This work was carried out independently, without institutional funding, without a research group, and without affiliation to any university or laboratory.

The author engaged eight large language models — Claude (Anthropic), Grok (xAI), ChatGPT (OpenAI), DeepSeek, Qwen (Alibaba), Gemini (Google), Kimi (Moonshot AI), and Mistral (Mistral AI) — as structured intellectual collaborators across 145 days of continuous development. Their contributions to architectural decisions, failure mode identification, debugging, test scenarios, and paper review are documented in the project’s commit history and ADRs. The *Test Driven Hardening* methodology documented in Section 6 emerged directly from this collaboration.

This paper discloses AI involvement openly and precisely. The author remained the final arbiter of all decisions; all empirical validation was performed by the human author.

The author wishes to acknowledge the friend whose business was destroyed by ransomware — who will recognize himself in these pages. And the Hospital Universitario de Badajoz, Extremadura, whose physicians diagnosed a rare disease and gave the author the years needed to build something worth building. Thank you Pedro Risco and Agustín Pijierro.

This work is dedicated to every hospital, school, and small organization that has ever been told that enterprise-grade security is beyond their reach.

References

- [1] Blake Anderson and David McGrew. Identifying encrypted malware traffic with contextual flow data. In *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 35–46, 2016.
- [2] Anthropic. Project glasswing: Securing critical software for the AI era. <https://www.anthropic.com/glasswing>, April 2026. Accessed: April 2026.
- [3] Black Fog. The state of ransomware 2025. <https://www.blackfog.com/>, 2025.
- [4] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2): 1153–1176, 2016.
- [5] Center for Internet Security. CIS Controls v8, 2021. URL <https://www.cisecurity.org/controls/v8>. Accessed: April 2026.
- [6] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000. doi: 10.1145/351240.351266.
- [7] Sebastian Garcia, Martin Grill, Jan Stiborek, and Alejandro Zunino. An empirical comparison of botnet detection methods. *Computers & Security*, 45:100–123, 2014.
- [8] IBM Security. Cost of a data breach report 2025, 2025.
- [9] INCIBE-CERT. Ciberataque ransomware paraliza actividad del Hospital Clínic de Barcelona. <https://www.incibe.es/>, 2023.
- [10] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, Internet Engineering Task Force, May 2010.
- [11] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Network and Distributed Systems Security Symposium (NDSS)*, 2018.
- [12] MITRE Corporation. MITRE Caldera: Automated adversary emulation platform. <https://caldera.mitre.org/>, 2024. Accessed: April 2026.

- [13] MITRE Corporation. CWE-22: Improper limitation of a pathname to a restricted directory ('Path Traversal'). <https://cwe.mitre.org/data/definitions/22.html>, 2024. Common Weakness Enumeration. Accessed: April 2026.
- [14] MITRE Corporation. CWE-367: Time-of-check time-of-use (TOCTOU) race condition. <https://cwe.mitre.org/data/definitions/367.html>, 2024. Common Weakness Enumeration. Accessed: April 2026.
- [15] MITRE Corporation. CWE-59: Improper link resolution before file access ('Link Following'). <https://cwe.mitre.org/data/definitions/59.html>, 2024. Common Weakness Enumeration. Accessed: April 2026.
- [16] MITRE Corporation. CWE-78: Improper neutralization of special elements used in an OS command ('OS Command Injection'). <https://cwe.mitre.org/data/definitions/78.html>, 2024. Common Weakness Enumeration. Accessed: April 2026.
- [17] Open Information Security Foundation. Suricata open source IDS/IPS/NSM engine. <https://suricata.io/>, 2010.
- [18] André Pinto et al. Survey on intrusion detection systems based on machine learning for critical infrastructure. *Sensors*, 23(5):2415, 2023.
- [19] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, pages 229–238, 1999.
- [20] Kostya Serebryany. libFuzzer – a library for coverage-guided fuzz testing. <https://llvm.org/docs/LibFuzzer.html>, 2016. LLVM Project. Accessed: April 2026.
- [21] Iman Sharafaldin, Ali Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.
- [22] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P)*, pages 305–316, 2010. doi: 10.1109/SP.2010.25.
- [23] Ken Thompson. Reflections on trusting trust. *Communications of the ACM*, 27(8):761–763, 1984. doi: 10.1145/358198.358210.
- [24] Wazuh, Inc. Wazuh: Open source security platform, 2024. URL <https://wazuh.com>. Accessed: April 2026.